CAREBRIDGE: RURAL HEALTHCARE SUPPORT PLATFORM

A Full Stack Web Application using the MERN Stack

(MongoDB, Express.js, React, Node.js)

Project Report submitted in partial fulfilment of the requirements for the course

Full Stack Development (Project 14)

B.Tech Artificial Intelligence and Data Science — Academic Year 2026–2027

Submitted by

Team Member Name	Roll Number	Role in Project
Akhilon S	24BAD008	Team Lead – Integration and Review
Jeevanandham K	24BAD047	Frontend – React
Jeevadharan G	24BAD046	Backend – Node/Express and Database
Gowtham S	24BAD028	Testing, Documentation and Admin Workflows

Under the guidance of

Dr. S. Rajasekaran, Assistant Professor II

Department of Artificial Intelligence and Data Science

Kumaraguru College of Technology, Coimbatore

November 2026

GitHub: https://github.com/jeevaa006/fullstack

Live website: https://carebridge.gva.dpdns.org

Abstract

Rural and underserved communities often struggle to find verified doctors, to know when a consultation is possible, and to keep track of their medicines. CareBridge is a secure, role-based full-stack web application that addresses this coordination problem. It serves three kinds of users: patients, who search an approved-only doctor directory, request exact appointment slots, manage personal medicine reminders, find published hospitals and read attributed educational health information; doctors, who maintain their professional profile and availability and respond to appointment requests; and administrators, who verify doctors, manage user accounts and the hospital directory, and view platform statistics. The application is built on the MERN stack: a React 18 single-page client built with Vite and React Router, an Express.js REST API running on Node.js, and a MongoDB database accessed through Mongoose, with HTML, CSS and JavaScript (ES6+) for the interface. Authentication uses bcrypt-hashed passwords and signed JSON Web Tokens stored in HttpOnly cookies, and every protected route re-checks the user's role and record ownership on the server. A partial unique MongoDB index prevents two active bookings for the same slot, even under concurrent requests. The patient interface is available in English and Tamil and is responsive across devices. The system is covered by automated API tests, was validated with a clean lint and production build, and is deployed on Vercel with MongoDB Atlas. The result is a working, demonstrable platform in which doctors are visible only after administrator approval, bookings are reliable, and the limits of the system are stated openly.

Keywords: MERN Stack, REST API, React, MongoDB, JWT Authentication, Role-Based Access Control

Table of Contents

Chapter 1: Introduction....1
1.1 Background....1
1.2 Problem Statement....1
1.3 Objectives....1
1.4 Scope....2
1.5 Existing Systems and Their Limitations....2
1.6 Organisation of the Report....3
Chapter 2: Requirement Analysis....4
2.1 Functional Requirements....4
2.2 Non-Functional Requirements....4
2.3 User Roles....5
2.4 Hardware and Software Requirements....5
Chapter 3: System Design....6
3.1 System Architecture....6
3.2 Use Case Diagram....6
3.3 Data Flow Diagram....7
3.4 Database Design....8
3.5 API Design....10
3.6 UI Design / Wireframes....11
Chapter 4: Implementation....13
4.1 Project Folder Structure....13
4.2 Frontend — HTML, CSS and JavaScript....13
4.3 Frontend — React....13
4.4 Backend — Node.js and Express.js....14
4.5 Database — MongoDB....14
4.6 Integration of Frontend and Backend....15
4.7 Key Code Snippets....15
4.8 Security Measures....17
Chapter 5: Deployment....18
5.1 Deployment Platforms....18
5.2 Deployment Steps....18
5.3 Version Control....19
5.4 Live Links....20
Chapter 6: Results and Discussion....21
6.1 Application Screenshots....21
6.2 Objectives Achieved....31
6.3 Performance Observations....32
6.4 Challenges Faced....32
Chapter 7: Conclusion and Future Scope....34
7.1 Conclusion....34
7.2 Future Scope....34
7.3 Individual Contribution....34
7.4 References....35

List of Figures

Figure 3.1: Three-tier system architecture of CareBridge....6
Figure 3.2: Use case diagram showing the four actors and their actions....7
Figure 3.3: Level 0 data flow diagram (context diagram)....7
Figure 3.4: Level 1 data flow diagram....8
Figure 3.5: Collections and relationships (entity-relationship view)....8
Figure 3.6: Wireframes: (a) login and registration, (b) role workspace with KPI cards, (c) data-table page....12
Figure 3.7: React component hierarchy....12
Figure 5.1: Vercel project overview: production deployment on the custom domain....19
Figure 5.2: Vercel deployment history; every push to main triggers a production build....19
Figure 5.3: GitHub repository jeevaa006/fullstack....20
Figure 6.1: Login page....21
Figure 6.2: Patient overview....21
Figure 6.3: Patient dashboard summary....22
Figure 6.4: Find a doctor....22
Figure 6.5: Hospitals and healthcare centres....23
Figure 6.6: Search healthcare services....23
Figure 6.7: My appointments....24
Figure 6.8: Medicine reminders....24
Figure 6.9: Health guidance....25
Figure 6.10: Healthcare news and research....25
Figure 6.11: Patient profile....26
Figure 6.12: Help and support....26
Figure 6.13: Doctor dashboard....27
Figure 6.14: Doctor profile editor....27
Figure 6.15: Availability management....28
Figure 6.16: Appointment requests....28
Figure 6.17: Administrator dashboard....29
Figure 6.18: User management....29
Figure 6.19: Doctor verification....30
Figure 6.20: Appointment management....30
Figure 6.21: Hospital management....31
Figure 6.22: Platform statistics....31

List of Tables

Table 1.1: Comparison of CareBridge with existing approaches....2
Table 2.1: Functional requirements....4
Table 2.2: User roles and permissions....5
Table 2.3: Hardware and software requirements....5
Table 3.1: Database schema (all collections include timestamps)....9
Table 3.2: REST API endpoints (prefix /api)....10
Table 4.1: Security measures implemented....17
Table 6.1: Status of objectives from Section 1.3....31
Table 6.2: Recorded measurements....32
Table 7.1: Individual contribution of team members....34

Chapter 1: Introduction

1.1 Background

Healthcare delivery in rural and semi-urban India is often limited less by the absence of doctors than by the difficulty of **finding and coordinating** care. Information about which doctor is qualified, which hospital offers a particular service, and whether a consultation slot is open is scattered across notice boards, phone calls and word of mouth. Digital directories and appointment platforms can reduce this friction, but they must also be trustworthy: a directory that lists unverified practitioners, or a booking flow that lets two patients claim the same slot, can do more harm than good.

CareBridge is a full-stack web application, built on the MERN stack (MongoDB, Express.js, React and Node.js), that addresses this coordination problem for rural and underserved communities. It is an academic demonstration of a secure, role-based support platform; it does not diagnose or treat, and every screen carries a medical disclaimer.

1.2 Problem Statement

Patients in rural communities struggle to locate verified doctors and nearby healthcare centres, to see when a doctor can actually be consulted, and to keep track of their own medicine schedules. Appointment coordination depends on phone calls during working hours, and health information found online is frequently unverified or unavailable in the patient's own language. Existing general-purpose platforms are not designed around administrator-verified listings or around the separation of patient, doctor and administrator data that a trustworthy rural service needs. What is missing is a single, secure application in which doctors are approved before they become visible, slots are exact and cannot be double-booked, and each role sees only the data it is entitled to.

1.3 Objectives

1. **Secure access:** allow patients and doctors to register and sign in using bcrypt-hashed passwords and JWT sessions delivered in HttpOnly cookies, with administrators created only through a protected seed script.
2. **Verified directory:** show only administrator-approved doctors and only published hospitals in public search results.
3. **Reliable booking:** let patients request exact doctor-published slots, enforce a constrained status lifecycle (Pending, Confirmed, Rejected, Cancelled, Completed), and make duplicate active bookings impossible even under concurrent requests.
4. **Personal adherence support:** let patients create, edit, deactivate and delete their own medicine reminders.

5. **Administration:** give administrators tools for doctor verification, user account status, hospital directory management and platform statistics.
6. **Accessible and deployable:** provide a responsive English/Tamil interface, attributed educational health information, automated tests, and a live Vercel deployment with documented setup.

1.4 Scope

In scope: registration, login and session restoration; three roles (patient, doctor, administrator); doctor verification; approved-only doctor search with filters, sorting and pagination; doctor-managed availability; appointment request, review and status transitions; medicine reminders; a published hospital directory with an optional Leaflet map; administrator statistics and user management; local, rule-based symptom guidance and attributed MedlinePlus, PubMed, news and OpenStreetMap information; English and Tamil text for the patient interface.

Out of scope: clinical diagnosis or triage, prescriptions, video consultation or chat, push/SMS/e-mail notifications, payments or insurance, government registry integration, recurring appointments, and a certified accessibility audit. No real patient data is used; all demonstration records are synthetic.

1.5 Existing Systems and Their Limitations

Table 1.1 compares CareBridge with three common approaches. The comparison is qualitative and describes typical characteristics of each category rather than any single named product.

Table 1.1: Comparison of CareBridge with existing approaches

Approach	Typical strengths	Limitations for rural use	CareBridge response
Phone calls / notice boards	No technology needed; familiar	Office hours only; no live slot view; no verification trail	Online slots with an auditable status lifecycle
Commercial doctor-booking aggregators	Large listings, polished booking flows	Urban focus; listings not tied to a local administrator review; limited local-language support	Admin-approved profiles only; Tamil patient interface
National / state telemedicine portals	Government backing; remote consultation	Consultation-centred rather than local directory and slot booking; no local reminders	Local directory, slot booking and reminders in one app

The gap CareBridge fills is a small, transparent platform where a responsible administrator controls what is publicly visible, the booking rules are enforced by the database rather than by the interface, and the limits of the system are stated openly.

1.6 Organisation of the Report

- **Chapter 1** introduces the problem, objectives, scope and related approaches.
- **Chapter 2** lists functional and non-functional requirements, user roles and the software environment.
- **Chapter 3** presents the architecture, use cases, data-flow diagrams, database design, REST API and UI design.
- **Chapter 4** describes the implementation of the frontend, backend, database, integration and security measures.
- **Chapter 5** covers deployment on Vercel and MongoDB Atlas and version control.
- **Chapter 6** presents application screenshots, objectives achieved, performance observations and challenges.
- **Chapter 7** concludes the report and outlines future scope and individual contributions.

Chapter 2: Requirement Analysis

2.1 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR1	The system shall allow a visitor to register as a patient or a doctor with validated name, e-mail and password; e-mail is normalised and unique.
FR2	The system shall allow users to sign in, sign out and restore their session after a page reload.
FR3	Administrator accounts shall be created only through a protected seed script, never through public registration.
FR4	Doctor registration shall create a profile in the pending state; only an administrator can approve or reject it.
FR5	The doctor directory shall list approved profiles only, with name, specialization, district and hospital filters, sorting and pagination.
FR6	A doctor shall be able to edit their profile and publish or remove future availability slots.
FR7	A patient shall be able to request an exact offered slot; a second active request for the same doctor, date and start time shall be rejected.
FR8	Appointments shall follow a constrained status lifecycle with separate transition rules for patient, doctor and administrator.
FR9	A patient shall be able to create, list, edit, deactivate and delete their own medicine reminders.
FR10	Signed-in users shall be able to search published hospitals and view optional map coordinates.
FR11	An administrator shall be able to create, edit, publish, unpublish and delete hospital records.
FR12	An administrator shall be able to view platform statistics, list users, activate or deactivate accounts, and view all appointments.
FR13	Users shall be able to read non-diagnostic educational guidance and search attributed MedlinePlus topics, news and PubMed citations.
FR14	The patient interface shall be available in English and Tamil, and the whole interface shall be responsive.

2.2 Non-Functional Requirements

- **Security:** passwords hashed with bcrypt (cost 12); JWT in an HttpOnly, SameSite=Lax cookie (Secure in production); Helmet headers; exact-origin CORS; rate-limited authentication; server-side role and ownership checks on every protected route.
- **Privacy:** passwords never returned by the API; patient records scoped to the signed-in user; doctor detail omits the account e-mail; external services never receive patient or account data.
- **Performance and scalability:** pagination capped at 50 items; JSON body limited to 32 kB; external calls time out and are cached; the API runs as a stateless Vercel Function with a cached database connection.
- **Reliability:** health endpoints separate service status from database status; database failures return HTTP 503 instead of crashing.

- **Usability and responsiveness:** a consistent layout across roles, status badges, empty and loading states, and an off-canvas navigation drawer on small screens.
- **Maintainability:** one documented API router, six focused Mongoose models, environment-based configuration and an automated test suite.

2.3 User Roles

Table 2.2: User roles and permissions

Role	Can do	Cannot do
Guest (visitor)	Open login, registration, privacy and help pages.	Open any workspace, directory or dashboard page; the client router redirects to login.
Patient	Search approved doctors and published hospitals; request and cancel own appointments; manage own reminders and profile; read guidance; switch language.	View other patients' data; approve doctors; manage hospitals; decide appointment requests.
Doctor	Maintain own profile and availability; confirm, reject or complete requests addressed to them; view history.	See other doctors' appointments; create reminders; verify doctors; manage hospitals.
Administrator	Verify or reject doctors; activate or deactivate users; manage hospitals; view statistics and all appointments.	Register through the public form; deactivate their own account.

2.4 Hardware and Software Requirements

Table 2.3: Hardware and software requirements

Category	Requirement	Version used
Operating system	Linux cloud workspace; any OS with a modern browser for end users	Linux (Ubuntu-based workspace)
Runtime	Node.js	20+ required; 22.x used (Dockerfile: node:22-alpine)
Package manager	npm	10.x
Frontend library	React, React DOM, Vite	React ^18.2.0; Vite ^5.0.8
Backend framework	Express.js	4.22.3
Database	MongoDB (Atlas)	Atlas cloud cluster
ODM	Mongoose	8.24.1
Editor / IDE	Visual Studio Code (cloud workspace)	Current stable
Tools	Git, GitHub, Vercel, Supertest, ESLint	Supertest ^7.3.1; ESLint ^8.55.0
Other libraries	Axios, React Router, Leaflet / React-Leaflet, Lucide React, bcryptjs, jsonwebtoken, Helmet, CORS, express-rate-limit, cookie-parser, fast-xml-parser, dotenv	Axios ^1.21.0; React Router ^7.18.5; Leaflet ^1.9.4; bcryptjs ^3.0.3; jsonwebtoken ^9.0.3; Helmet ^8.3.0; express-rate-limit ^8.7.1; fast-xml-parser ^5.11.2
Hardware	Any machine able to run a browser; development used a cloud workspace	No special hardware

Chapter 3: System Design

3.1 System Architecture

CareBridge follows the three-tier architecture shown in Figure 3.1. The React single-page application runs in the browser and calls a JSON REST API under `/api/v1` through a shared Axios instance with credentials enabled, so the session cookie is attached automatically. The Express application applies security middleware, authenticates the request, checks the role, validates input and queries MongoDB through Mongoose. Optional educational data is fetched only by the server from external public services, with timeouts, caching and rate limiting, so no patient data leaves the system.

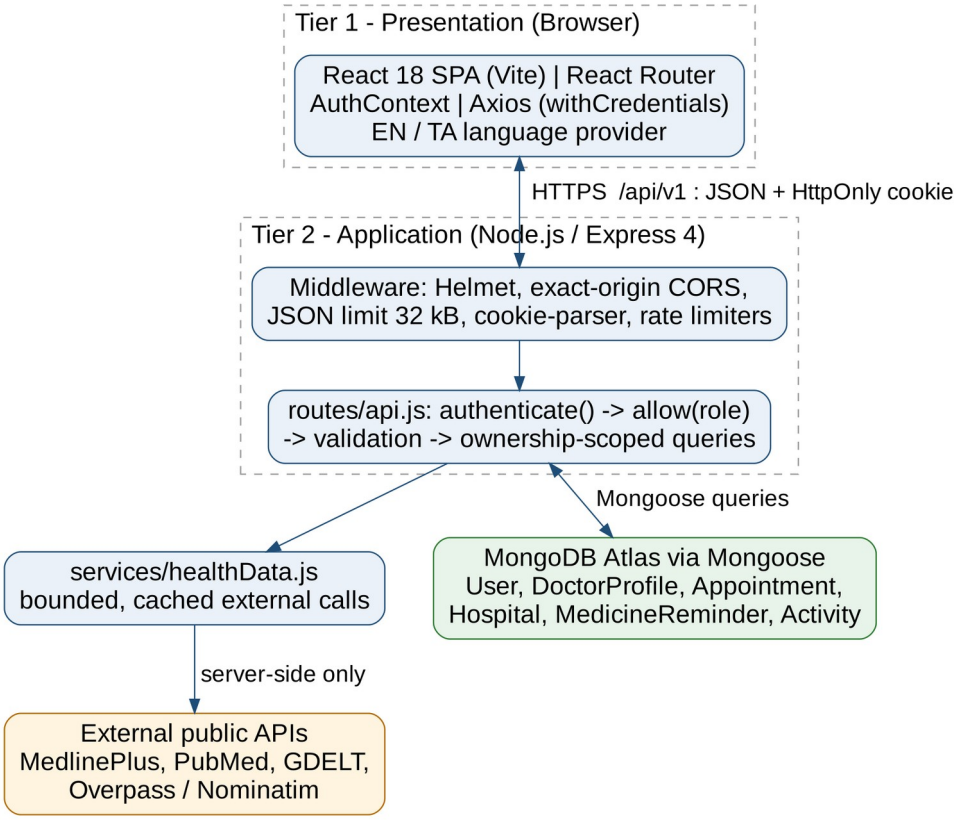


Figure 3.1: Three-tier system architecture of CareBridge

A typical request proceeds as follows: (1) the client sends the request with the HttpOnly cookie; (2) Helmet, CORS and the body parser run; (3) the `authenticate` middleware verifies the JWT and reloads the user from the database; (4) the `allow` middleware checks the role; (5) the route validates input and applies ownership filters inside the database query; (6) the response is returned as JSON with a meaningful HTTP status code.

3.2 Use Case Diagram

Figure 3.2 shows the actors and the actions each can perform. The administrator is the only actor who can change what the public sees, which underpins the trust model of the platform.

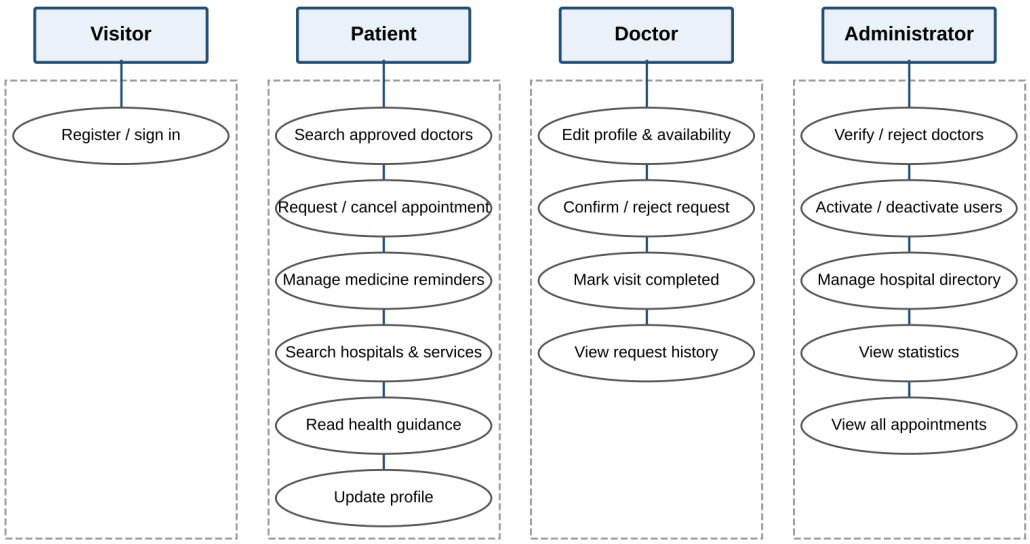


Figure 3.2: Use case diagram showing the four actors and their actions

3.3 Data Flow Diagram

Figure 3.3 is the Level 0 (context) diagram: the whole system is one process exchanging data with the three user roles and with external health sources. Figure 3.4 expands it to Level 1, showing the seven major processes and the five principal data stores.

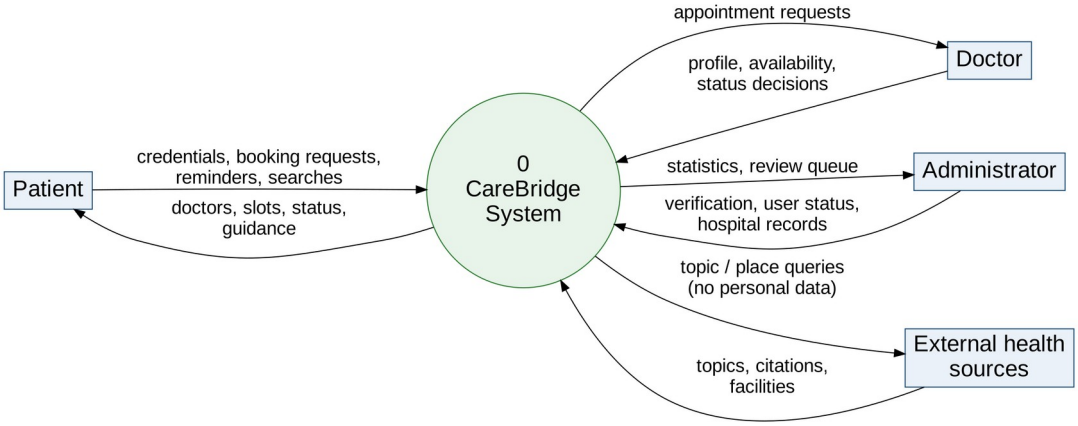


Figure 3.3: Level 0 data flow diagram (context diagram)

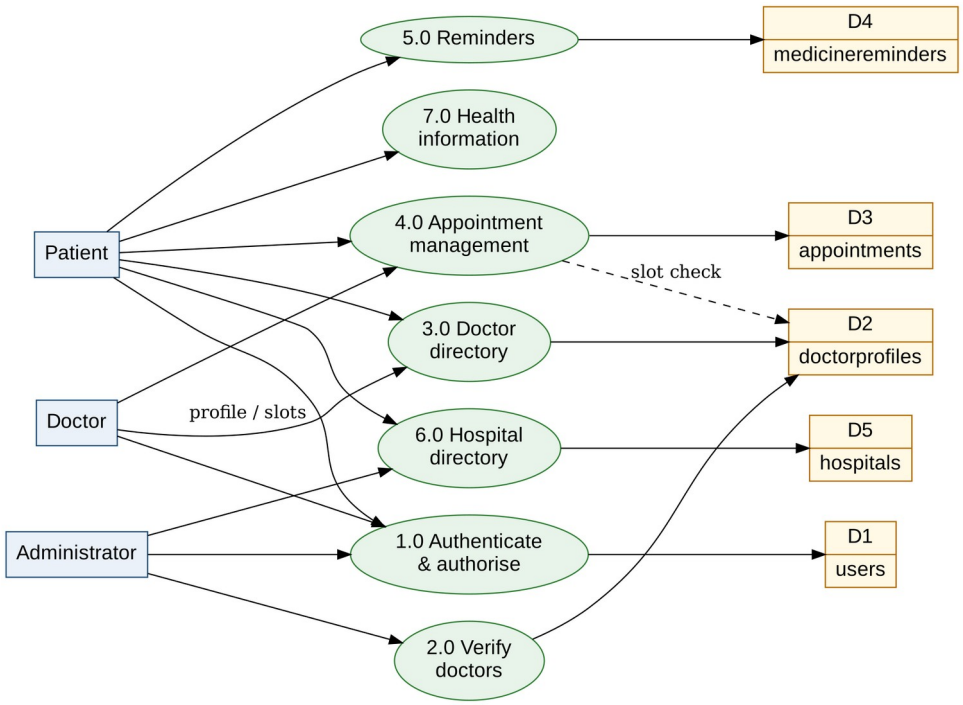


Figure 3.4: Level 1 data flow diagram

3.4 Database Design

CareBridge stores its data in six MongoDB collections managed through Mongoose. Figure 3.5 shows their relationships. A `DoctorProfile` references its `User` (one-to-one); `Appointment` references the patient and the doctor, both `User` documents; `MedicineReminder` references its patient; and `Activity` records sign-ins and administrator actions. Doctor availability slots are **embedded** inside the profile because they are always read and replaced together with it, whereas appointments are **referenced** because they are queried independently by patient, doctor and status. Hospitals are standalone directory records.

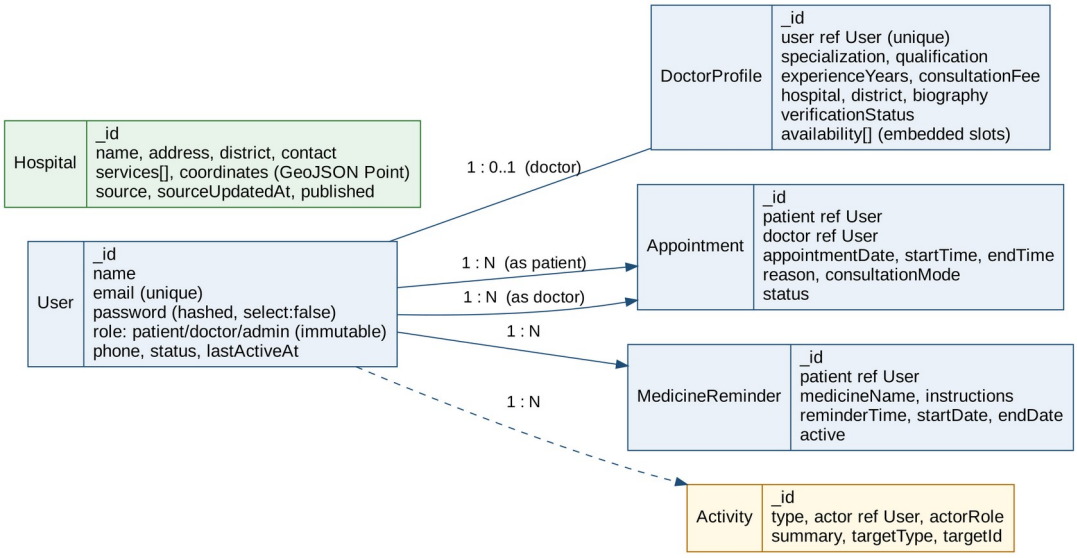


Figure 3.5: Collections and relationships (entity-relationship view)

Table 3.1: Database schema (all collections include timestamps)

Collection	Field	Type	Constraints	Description
users	name	String	required, 2–100 chars	Display name
	email	String	required, unique, lowercase	Login identity
	password	String	required, min 8, select:false	bcrypt hash (cost 12)
	role	String	enum patient/doctor/admin, immutable	Authorisation role
	phone, status	String	status: active/inactive	Contact; account state
	lastActiveAt	Date	indexed	Presence timestamp
doctorprofiles	user	ObjectId → User	required, unique	Owning account
	specialization, qualification	String	required	Professional details
	experienceYears	Number	required, 0–80	Experience
	consultationFee	Number	required, 0–100000	Fee (INR)
	hospital, district, biography	String	district required	Practice location
	verificationStatus	String	enum pending/approved/rejected	Admin review state
	availability[]	Array of {date, startTime, endTime}	regex-validated	Embedded slots
appointments	patient, doctor	ObjectId → User	required, indexed	Participants
	appointmentDate	String	required, YYYY-MM-DD	Visit day
	startTime, endTime	String	required, HH:MM	Slot times
	reason	String	required, 3–500 chars	Reason for visit
	consultationMode	String	enum in-person/online	Mode (request only)
	status	String	enum Pending/Confirmed/Rejected/Cancelled/Completed	Lifecycle state
medicinereminders	patient	ObjectId → User	required, indexed	Owner
	medicineName, instructions	String	name required, ≤120	Reminder text
	reminderTime	String	required, HH:MM	Daily time
	startDate, endDate	String	required; end ≥ start	Active period
	active	Boolean	default true	Enabled flag
hospitals	name, address, district	String	required	Directory entry
	contact, services[]	String / [String]	optional	Contact and services
	coordinates	GeoJSON Point	optional, [lon, lat]	Map location
	source, sourceUpdatedAt	String / Date	default “CareBridge administrator”	Data provenance
	published	Boolean	default false	Public visibility
activities	type, actorRole, summary	String	type enum, indexed	Audit event
	actor, targetId	ObjectId	optional	Who / what

Three indexes carry integrity guarantees: a unique index on `users.email`, a unique index on `doctorprofiles.user`, and a **partial unique index** on `appointments` over `(doctor, appointmentDate, startTime)` that applies only to documents whose status is Pending or Confirmed. Rejected, Cancelled and Completed appointments therefore release their slot automatically.

3.5 API Design

All endpoints are served under the `/api` prefix and return JSON. Errors use the shape `{ "error": "message" }` with standard status codes. “Any” in the Auth column means any signed-in role. Public directory endpoints return approved or published records only; the login-first rule is enforced by the client router.

Table 3.2: REST API endpoints (prefix /api)

Method	Endpoint	Description	Auth	Request body / query
GET	/health	Service health	No	—
GET	/health/db	MongoDB ping (200 / 503)	No	—
POST	/v1/auth/register/patient	Register patient; set cookie	No	name, email, password, phone?
POST	/v1/auth/register/doctor	Register doctor with pending profile	No	name, email, password, specialization, qualification, experienceYears, consultationFee, district, …
POST	/v1/auth/login	Sign in; set cookie	No	email, password
POST	/v1/auth/logout	Clear session cookie	No	—
GET	/v1/auth/me	Current user	Any	—
PATCH	/v1/auth/profile	Update name / phone	Any	name?, phone?
POST	/v1/presence/heartbeat	Record activity timestamp	Any	—
GET	/v1/doctors	Approved doctors (filter, sort, page)	No	query: name, specialization, district, hospital, sort, page
GET	/v1/doctors/:id	Approved doctor detail	No	—
GET	/v1/doctors/:id/availability	Future slots with reserved flag	No	—
GET	/v1/doctors/me/profile	Own doctor profile	Doctor	—
PATCH	/v1/doctors/me/profile	Update own profile	Doctor	profile fields
GET	/v1/doctors/me/availability	Own slots	Doctor	—
PUT	/v1/doctors/me/availability	Replace own slots	Doctor	slots[{date, startTime, endTime}]
POST	/v1/doctors/sync-references	Maintenance: sync doctor references	Admin	—
POST	/v1/appointments	Request a slot	Patient	doctorId, appointmentDate, startTime, reason, consultationMode?
GET	/v1/appointments	List own (admin: all) appointments	Any	query: status, page
GET	/v1/appointments/:id	Appointment detail (ownership checked)	Any	—
PATCH	/v1/appointments/:id/status	Change status per role map	Any	status

Method	Endpoint	Description	Auth	Request body / query
GET	/v1/hospitals	Published hospitals (search, page)	No	query: q, district, service
GET	/v1/hospitals/:id	Published hospital detail	No	—
GET	/v1/hospitals/nearby	Nearby OpenStreetMap facilities	No	query: q or lat, lon
GET	/v1/admin/hospitals	All hospital records	Admin	—
POST	/v1/admin/hospitals	Create hospital	Admin	name, address, district, services?, coordinates?
PATCH	/v1/admin/hospitals/:id	Edit / publish hospital	Admin	name?, address?, published?, …
DELETE	/v1/admin/hospitals/:id	Delete hospital	Admin	—
GET	/v1/reminders	Own reminders	Patient	—
POST	/v1/reminders	Create reminder	Patient	medicineName, reminderTime, startDate, endDate, instructions?
PATCH	/v1/reminders/:id	Edit / deactivate reminder	Patient	any reminder field, active
DELETE	/v1/reminders/:id	Delete own reminder	Patient	—
GET	/v1/admin/stats	Platform counts	Admin	—
GET	/v1/admin/analytics[/live]	Aggregated analytics (rate-limited)	Admin	query: range, granularity
GET	/v1/admin/users	List users (search, role, status)	Admin	query
PATCH	/v1/admin/users/:id/status	Activate / deactivate user	Admin	status
GET	/v1/admin/doctors	Doctors by verification status	Admin	query: status
PATCH	/v1/admin/doctors/:id/verification	Approve / reject doctor	Admin	status
GET	/v1/admin/appointments	All appointments	Admin	query: status
GET	/v1/health-information/search	MedlinePlus topic search	No	query: q
POST	/v1/health-information/guidance	Non-diagnostic guidance	No	symptoms
GET	/v1/news/medical	Aggregated health news (GDELT)	No	query: topic
GET	/v1/research	PubMed citations	No	query: query

3.6 UI Design / Wireframes

The interface uses one shared shell for all roles: a header with the brand, language selector (patients only) and account menu; a role-specific sidebar that becomes an off-canvas drawer on small screens; and a content area built from page headings, cards, forms and tables. Figure 3.6 shows the wireframes of the three layout patterns, and Figure 3.7 shows the React component hierarchy.

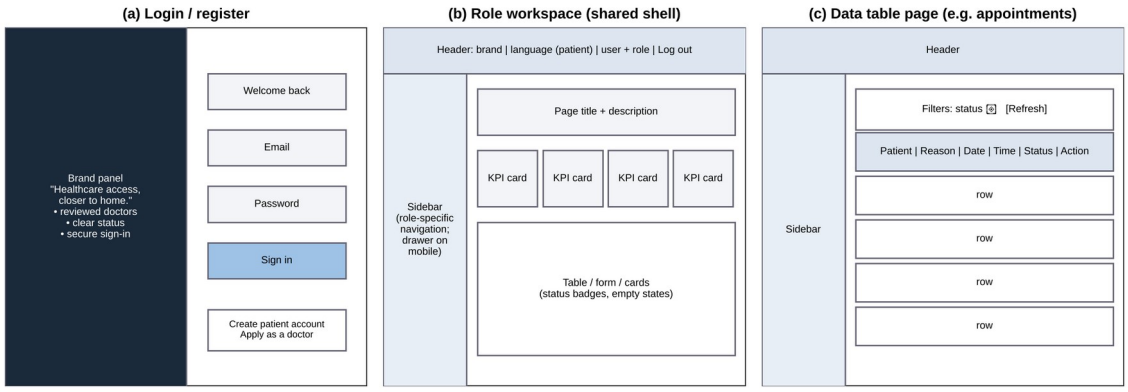


Figure 3.6: Wireframes: (a) login and registration, (b) role workspace with KPI cards, (c) data-table page

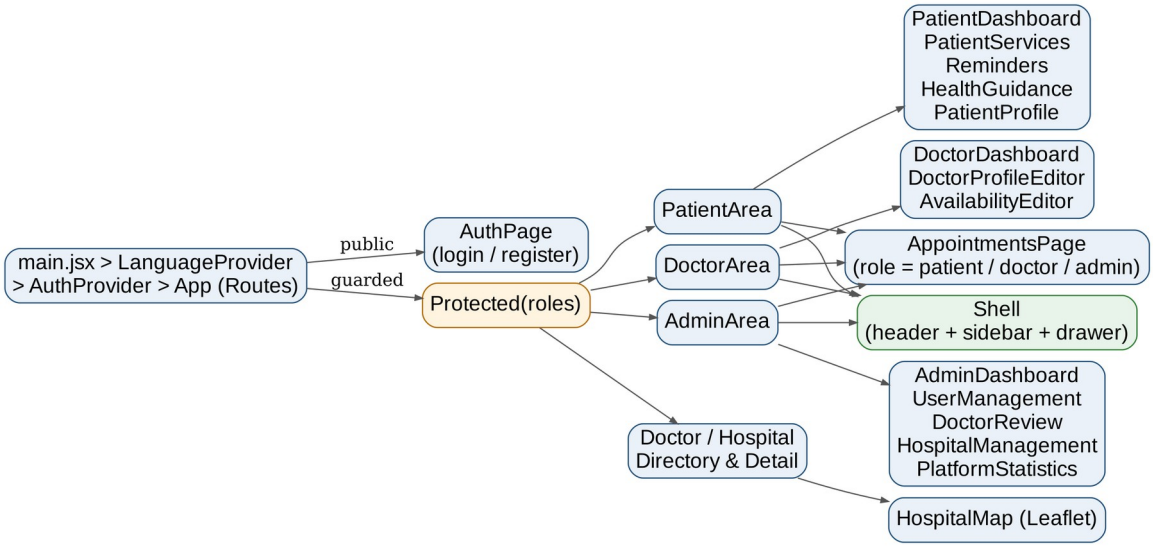


Figure 3.7: React component hierarchy

Chapter 4: Implementation

4.1 Project Folder Structure

The repository keeps the Express API at the project root and the React client in `client/`, which allows a single Vercel project and a single `npm start` in production.

```
fullstack/
|-- index.js           # Express app: Helmet, CORS, DB guard
|-- routes/api.js      # All REST routes, auth, role checks
|-- models/            # user, doctorProfile, appointment,
|                      #   hospital, medicineReminder, activity
|-- lib/               # database.js, analytics, presence, activity
|-- services/          # healthData.js (MedlinePlus, PubMed, OSM)
|-- scripts/           # seedAdmin, seedDatabase, dbBackup
|-- tests/             # api, language and smartEnter tests
|-- docs/              # architecture, API, database, deployment
|-- vercel.json, Dockerfile, .env.example
`-- client/
    |-- index.html     # Vite entry (semantic HTML shell)
    `-- src/
        |-- App.jsx, App.css, main.jsx
        |-- components/   # HospitalMap (Leaflet)
        |-- context/      # AuthContext (session state)
        |-- services/     # api.js (shared Axios instance)
        |-- i18n/         # translations, LanguageProvider
        `-- smartEnter.js, useSmartEnter.jsx
```

Listing 4.1: Directory structure of the repository

4.2 Frontend — HTML, CSS and JavaScript

The single `index.html` provides a semantic shell (`lang`, viewport, description and theme-colour metadata and one mount element); all views are rendered by React. Styling is a hand-written CSS design system in `App.css` (about 950 lines) using CSS custom properties for colours, spacing and typography, Flexbox and Grid for layout, and several media-query breakpoints (including 1100, 900, 720 and 520 px) that collapse the sidebar into an off-canvas drawer and stack cards and tables on narrow screens. Fonts (Instrument Sans, Newsreader and Noto Sans Tamil) are self-hosted through Fontsource so the interface renders identically offline and under the strict production Content Security Policy.

The JavaScript uses ES6+ throughout: modules, arrow functions, destructuring, optional chaining, template literals and `async/await` for all network calls. Client-side validation checks required fields, e-mail format and date ranges before submission, and the server repeats every check. A small custom hook, `useSmartEnter`, moves focus to the next field when Enter is pressed and submits only from the final field; its pure logic is unit-tested without a browser.

4.3 Frontend — React

The client is a React 18 single-page application built with Vite. `main.jsx` mounts `LanguageProvider` and `AuthProvider` around `App`, which declares all routes with React

Router 7 (Figure 3.7). Three role areas (`PatientArea`, `DoctorArea`, `AdminArea`) each declare nested routes inside the shared shell, and one `AppointmentsPage` component serves all three roles with a `role` prop.

- **State management:** `useState` and `useEffect` for page state; the Context API for authentication (`AuthContext`) and language (`LanguageProvider`). No Redux is needed.
- **Session restoration:** on load, `AuthProvider` calls `GET /auth/me`; while it is pending the app shows a loading screen, and it re-validates after back-forward cache restores so a signed-out user cannot reach cached pages.
- **Protected routes:** the `Protected` wrapper redirects unauthenticated users to `/login` (remembering the target) and users with the wrong role to `/access-denied`.
- **Forms and API calls:** controlled inputs, inline error alerts built from `errorMessage(error)`, and Axios calls through one shared instance (Section 4.6).
- **Internationalisation:** English and Tamil dictionaries with a unit test asserting that both expose identical keys and that Tamil strings keep their placeholders; only patient accounts can switch language, and the choice is stored per account.

4.4 Backend — Node.js and Express.js

`index.js` builds the Express application: it disables `x-powered-by`, applies Helmet (with a strict Content Security Policy in production), an exact-origin CORS allow-list with credentials, a 32 kB JSON body limit and `cookie-parser`. A database-readiness middleware opens a cached Mongoose connection lazily on the first `/api/v1` request — essential because Vercel imports the Express app as a function instead of running a listener. Unknown `/api` paths return a JSON 404, a central error handler returns generic 500 responses without stack traces, and the app serves `client/dist` with an SPA fallback in single-server mode.

`routes/api.js` holds all REST endpoints listed in Table 3.2. Authentication uses `authenticate` and `allow(...roles)` composed by `auth(...)` (Listing 4.3). Four rate limiters protect authentication (20 per 15 minutes), health-information (20 per minute), external data (10 per minute) and analytics (40 per minute). Input is validated with explicit checks (e-mail regular expression, password length, calendar-valid dates, time format, ObjectId validity), and user-supplied search text is regex-escaped before it reaches a query.

4.5 Database — MongoDB

MongoDB Atlas hosts the data. `lib/database.js` caches the connection on `globalThis` so warm serverless invocations reuse it, applies bounded timeouts (8 s server selection, 15 s connect, 30 s socket, pool size 10), retries after a short cool-down and redacts credentials from every log line. Mongoose schemas (Section 3.4) enforce types, enums, lengths and regular-

expression formats; a `pre-validate` hook rejects reminders whose end date precedes the start date; and `runValidators: true` is used on updates. CRUD is implemented with `create`, `find`, `findOneAndUpdate`, `findOneAndDelete` and `countDocuments`; the statistics endpoint runs nine counts in parallel with `Promise.all`, and the analytics module uses MongoDB aggregation pipelines over stored records only. Indexes are declared on the schemas (Section 3.4).

4.6 Integration of Frontend and Backend

The client never builds absolute URLs: a single Axios instance targets `/api/v1` with `withCredentials: true`. In development, Vite proxies `/api` to Express on port 5000; on Vercel, `vercel.json` rewrites `/api/*` to the Express service. The JWT is **not** stored by JavaScript — the server sets it as an HttpOnly cookie, so the browser attaches it automatically and scripts cannot read it. API errors arrive as `{ error }` and are displayed through one helper, `errorMessage`. On a 401 the user is returned to the login page; on a 409 (for example, a slot already requested) the message is shown inline so the patient can choose another slot.

4.7 Key Code Snippets

Password hashing

```
UserSchema.pre('save', async function hashPassword() {
  if (this.isModified('password'))
    this.password = await bcrypt.hash(this.password, 12);
});
UserSchema.methods.comparePassword = function (candidate) {
  return bcrypt.compare(candidate, this.password);
};
```

Listing 4.2: Mongoose pre-save hook hashing passwords (models/user.js)

Passwords are hashed with a cost factor of 12 whenever they change; the `password` field is also declared `select: false`, so it is never loaded unless explicitly requested at login.

Authentication and authorisation middleware

```
async function authenticate(req, res, next) {
  const token = req.cookies?.carebridge_token;
  if (!token) return fail(res, 401, 'Please sign in to continue.');
  const payload = jwt.verify(token, getJwtSecret());
  const user = await User.findById(payload.sub);
  if (!user || user.status !== 'active')
    return fail(res, 401, 'This account is unavailable.');
  req.user = user;          // role is re-read from the database
  next();
}
const allow = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next()
    : fail(res, 403, 'You do not have permission to do that.');
const auth = (...roles) =>
  [authenticate, ...(roles.length ? [allow(...roles)] : [])];
```

Listing 4.3: Authentication and role middleware (routes/api.js, abridged)

The token carries only the user id. Because the user and role are reloaded on every request, deactivating an account or changing access takes effect immediately, and a forged role claim is impossible.

Preventing double booking

```
AppointmentSchema.index(
  { doctor: 1, appointmentDate: 1, startTime: 1 },
  { unique: true,
    partialFilterExpression:
      { status: { $in: ['Pending', 'Confirmed'] } } },
);
```

Listing 4.4: Partial unique index on active appointments (models/appointment.js)

Only Pending and Confirmed appointments take part in the unique key, so cancelled or rejected requests free the slot. The route catches MongoDB error code 11000 and answers HTTP 409; because the check happens inside the database, concurrent requests cannot both succeed.

Constrained status transitions

```
const transitions = {
  patient: { Pending: ['Cancelled'], Confirmed: ['Cancelled'] },
  doctor:  { Pending: ['Confirmed', 'Rejected'],
             Confirmed: ['Completed'] },
  admin:   { Pending: ['Confirmed', 'Rejected', 'Cancelled'],
             Confirmed: ['Rejected', 'Cancelled', 'Completed'] },
};
if (!(transitions[req.user.role][appointment.status] || [])
      .includes(nextStatus))
  return fail(res, 409, 'This appointment status change is not allowed.');
```

Listing 4.5: Role-based status transition map (routes/api.js)

A single lookup table defines every legal state change per role. Ownership is checked first (a patient or doctor must be a party to the appointment), then the transition, so illegal actions return 403 or 409.

Protected routes in React

```
function Protected({ roles, children }) {
  const { user, loading } = useAuth();
  const location = useLocation();
  if (loading) return <Loading />;
  if (!user) return <Navigate to="/login" replace
                       state={{ from: location }} />;
  return roles && !roles.includes(user.role)
    ? <Navigate to="/access-denied" replace /> : children;
}
```

Listing 4.6: Role-aware route guard (client/src/App.jsx)

This component implements the login-first behaviour. It is a usability layer only: the real security boundary remains the server-side checks in Listing 4.3.

Session cookie and API client

```
// server: routes/api.js
res.cookie('carebridge_token', token, {
  httpOnly: true, sameSite: 'lax', path: '/',
  secure: process.env.NODE_ENV === 'production',
  maxAge: 7 * 24 * 60 * 60 * 1000,
});
// client: services/api.js
const api = axios.create({ baseURL: '/api/v1',
                           withCredentials: true });
```

Listing 4.7: Cookie issuance and shared Axios client

The seven-day token lives in an HttpOnly, SameSite=Lax cookie that is marked Secure in production, reducing exposure to cross-site scripting and cross-site request forgery.

4.8 Security Measures

Table 4.1: Security measures implemented

Measure	Implementation
Secrets	MONGODB_URI and JWT_SECRET (≥ 32 characters) are environment variables; `.env` is git-ignored; the server refuses to issue tokens without a valid secret (HTTP 503).
Password hashing	bcryptjs, cost 12; password field excluded from queries and JSON output.
Token expiry and storage	JWT valid for 7 days in an HttpOnly, SameSite=Lax, Secure (production) cookie.
Authorisation	Server-side role checks and ownership-scoped queries on every protected route; role is immutable.
Input validation / sanitisation	Route-level checks plus Mongoose validators; regex-escaped search input; 32 kB body limit; XML parser with entity processing disabled for MedlinePlus.
CORS and headers	Exact-origin allow-list with credentials; Helmet with a strict production CSP, frame-ancestors none and no `x-powered-by`.
Abuse protection	Rate limits on authentication, external-data, health-information and analytics endpoints.
Privacy	Doctor details omit e-mail; pending and rejected profiles hidden; external services never receive patient data.

Chapter 5: Deployment

5.1 Deployment Platforms

CareBridge is deployed on **Vercel** (Hobby plan) from the GitHub repository, with the database on **MongoDB Atlas**. `vercel.json` declares two services: an Express service (`index.js`) that runs as a Vercel Function and serves `/api/*`, and a Vite service (`client`) that serves the React build from the CDN. The application is reachable at the custom domain **https://carebridge.gva.dpdns.org**. A `Dockerfile` (Node 22 Alpine, two-stage build) is also provided for container hosting.

5.2 Deployment Steps

1. Create a MongoDB Atlas cluster, a database user, and allow network access for the serverless function.
2. Push the repository to GitHub and link it to a Vercel project (Git integration); Vercel installs dependencies and builds `client/dist` using the commands in `vercel.json`.
3. Add the environment variables `MONGODB_URI`, `JWT_SECRET` (random, at least 32 characters) and `NODE_ENV=production` for Preview and Production, then redeploy so they take effect.
4. Because the frontend and API share one origin, no CORS change is needed; for a separate frontend domain, add its exact origin to `FRONTEND_ORIGIN`.
5. Create the first administrator with `npm run seed:admin` (using `ADMIN_NAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`), then verify `/api/health`, `/api/health/db`, sign-in and a direct refresh on `/patient`.

The rewrite rules in `vercel.json` send `/api/*` to the Express service and every other path to the client service, whose own rewrite returns `index.html`, so deep links such as `/doctors/:id` survive a browser refresh. Figures 5.1 and 5.2 show the production deployment and its history on Vercel.

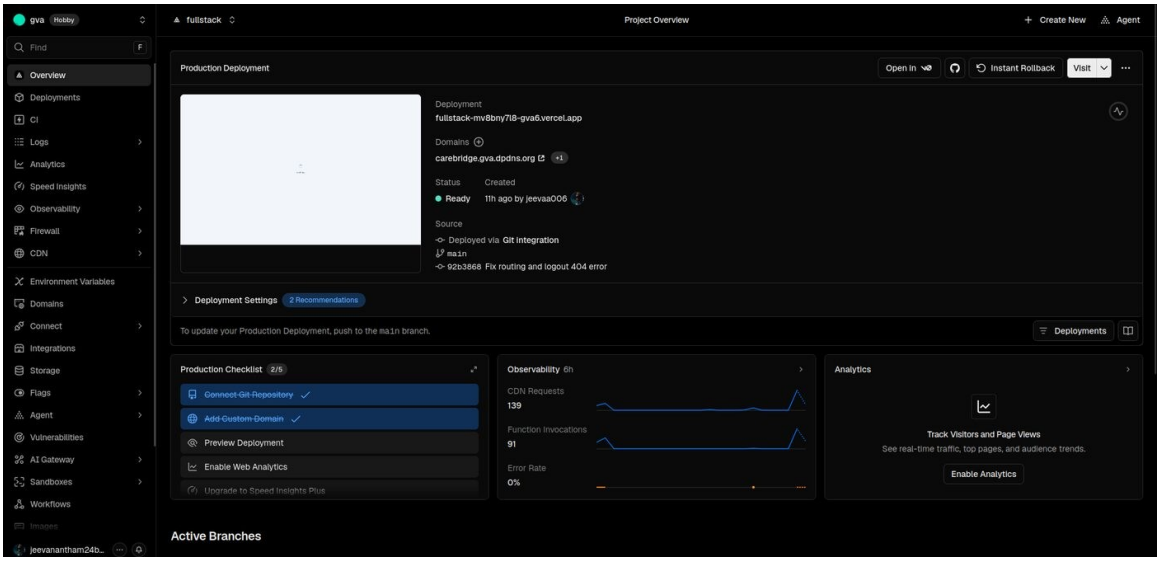


Figure 5.1: Vercel project overview: production deployment on the custom domain

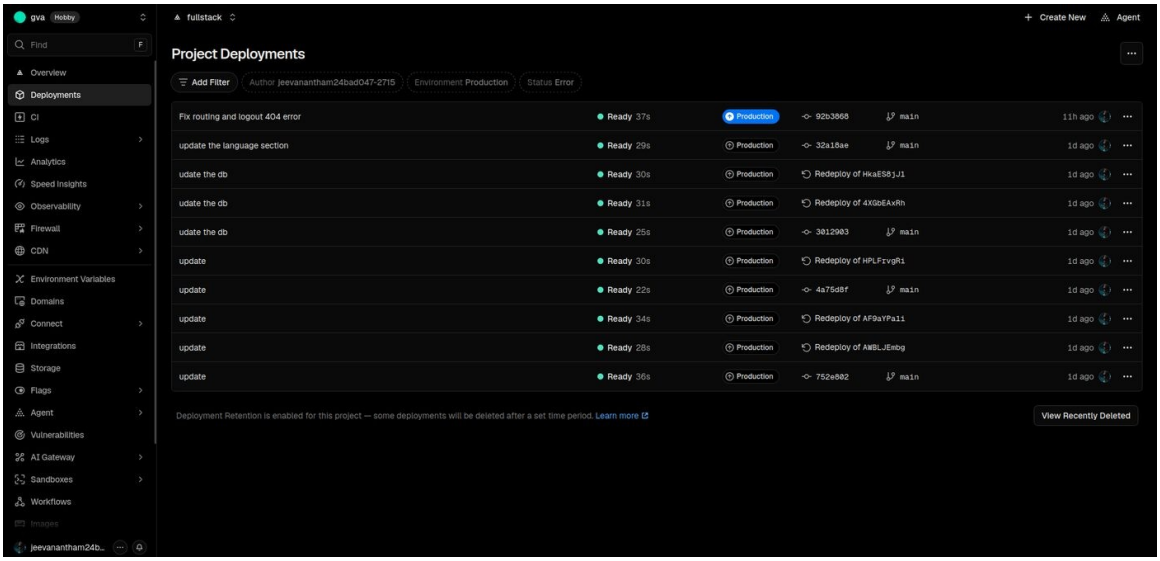


Figure 5.2: Vercel deployment history; every push to main triggers a production build

5.3 Version Control

The project is managed with Git and hosted on GitHub on a single `main` branch. Each push to `main` triggers an automatic Vercel production deployment, so the history of commits doubles as a deployment history (Figure 5.2). Commits are small and described by purpose (for example “Fix routing and logout 404 error” and “update the language section”). Secrets are never committed: `.env` is ignored and `.env.example` documents every variable. The repository README contains the full setup steps. Figure 5.3 shows the repository.

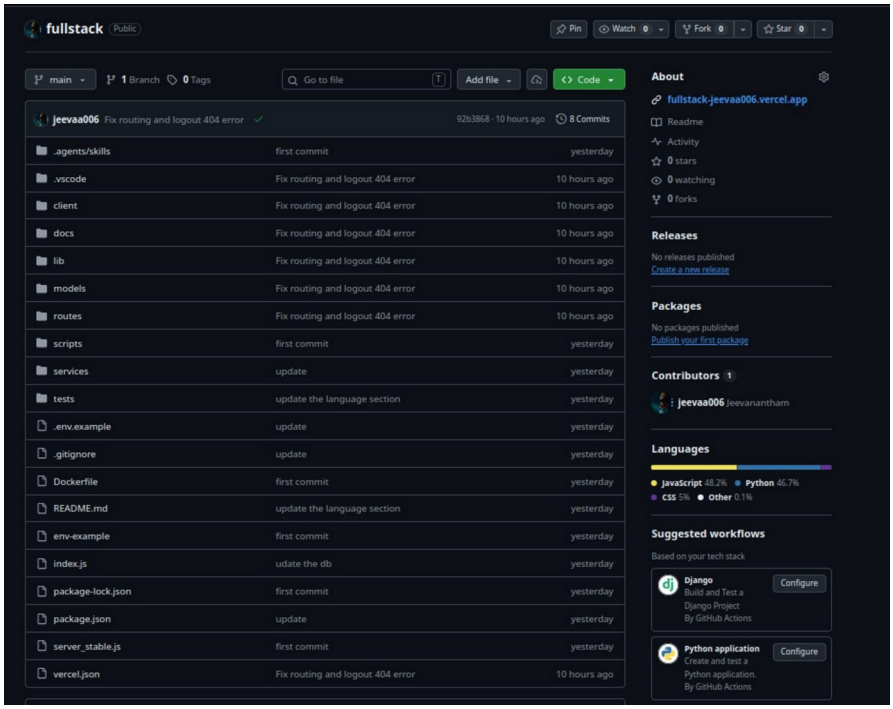


Figure 5.3: GitHub repository jeevaa006/fullstack

5.4 Live Links

GitHub repository: https://github.com/jeevaa006/fullstack

Live application: https://carebridge.gva.dpdns.org

Chapter 6: Results and Discussion

6.1 Application Screenshots

This section presents the main screens captured from the deployed application, grouped by role.

Authentication and patient screens

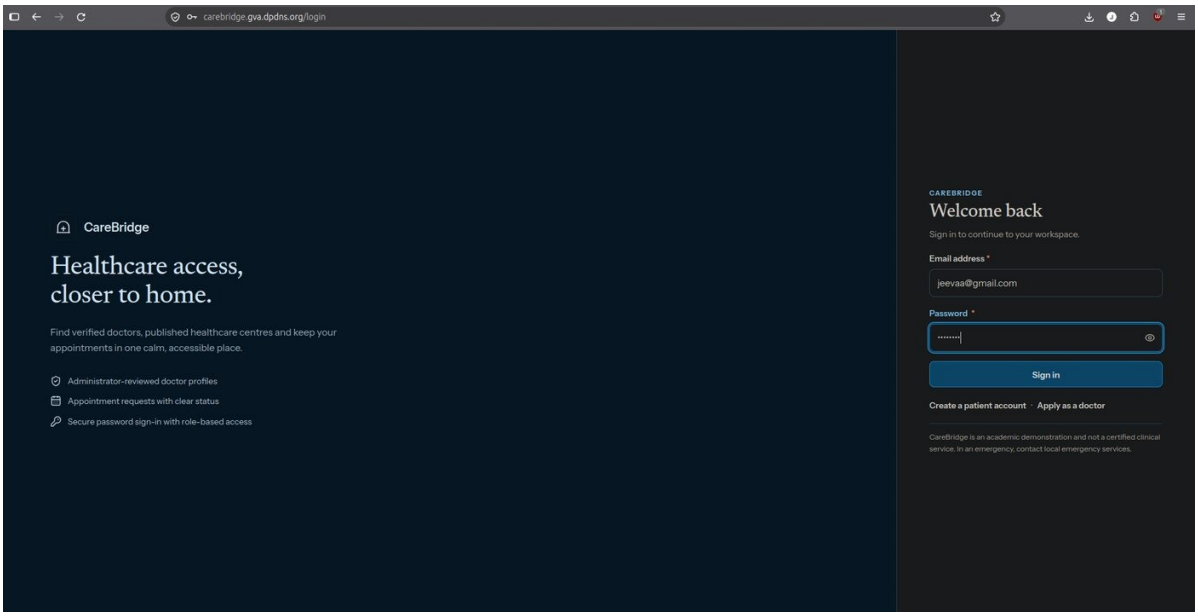


Figure 6.1: Login page

The login-first landing page: a brand panel on the left and the sign-in form on the right, with links to create a patient account or apply as a doctor and the medical disclaimer at the bottom.

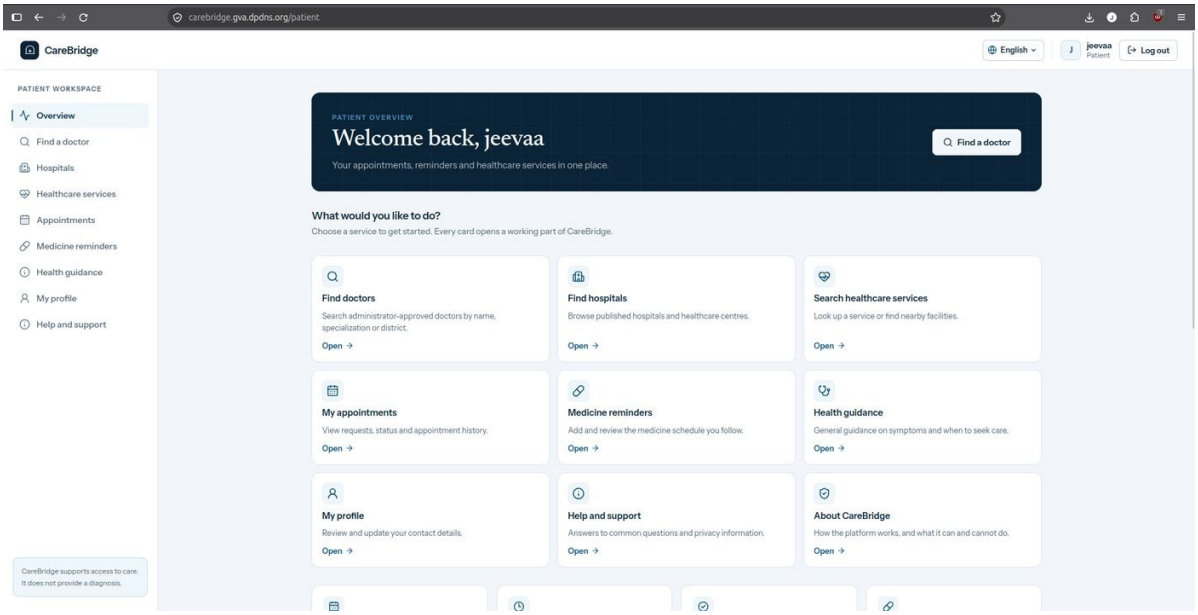


Figure 6.2: Patient overview

After sign-in the patient sees task cards for finding doctors and hospitals, searching healthcare services, appointments, reminders, guidance, profile and help, with the language selector in the header.

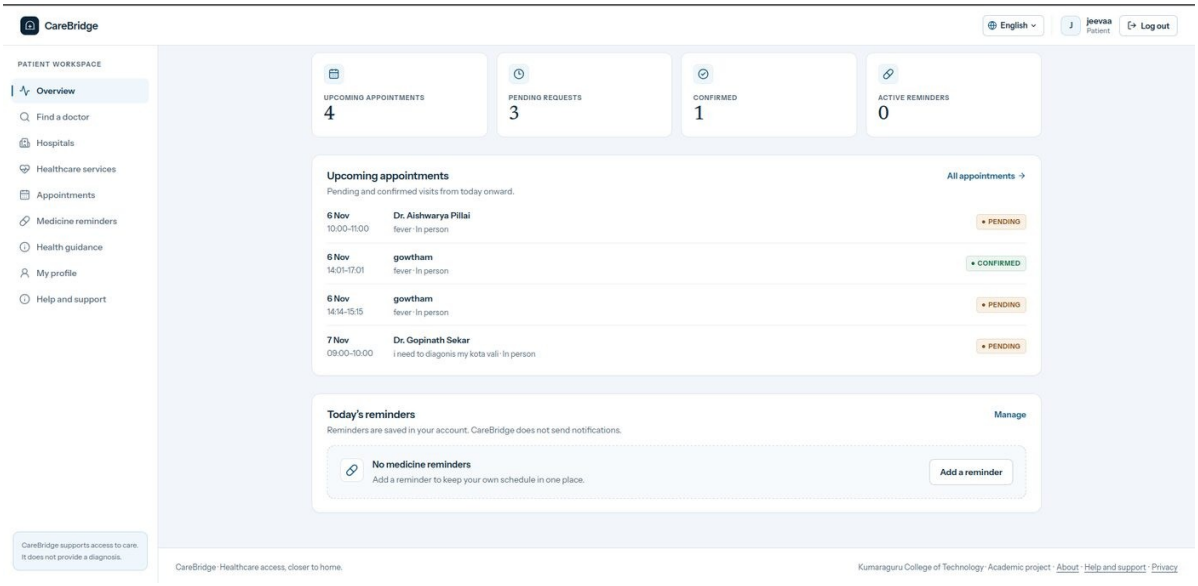


Figure 6.3: Patient dashboard summary

Counters for upcoming appointments, pending requests, confirmed visits and active reminders, followed by a list of upcoming appointments and today's reminders.

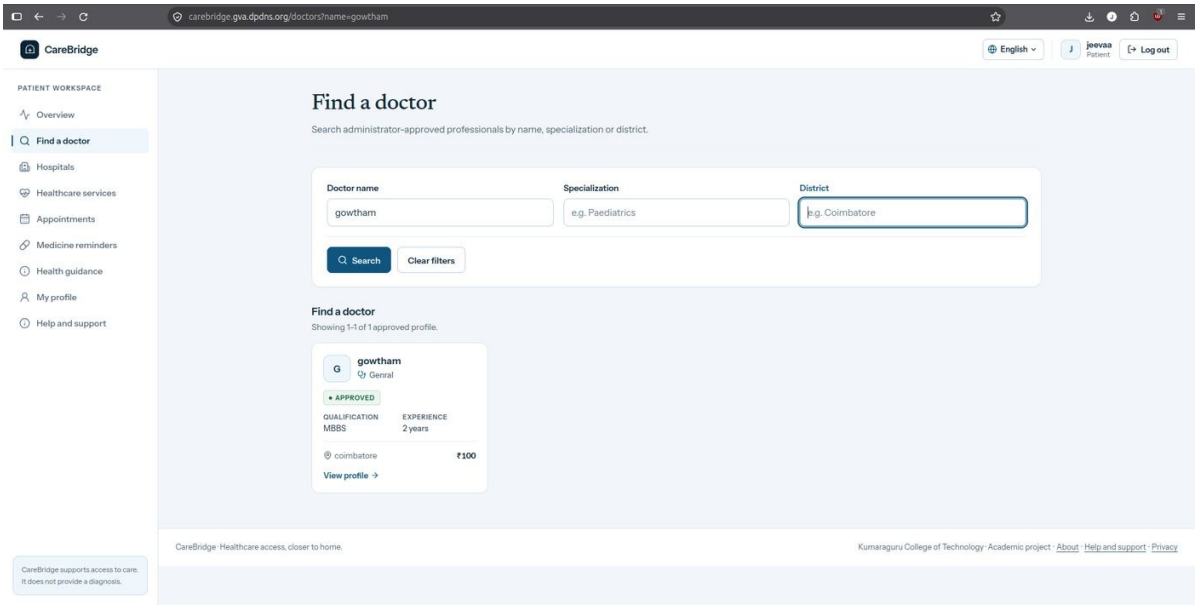


Figure 6.4: Find a doctor

Approved doctors can be searched by name, specialization or district; each result card shows verification status, qualification, experience, district and fee.

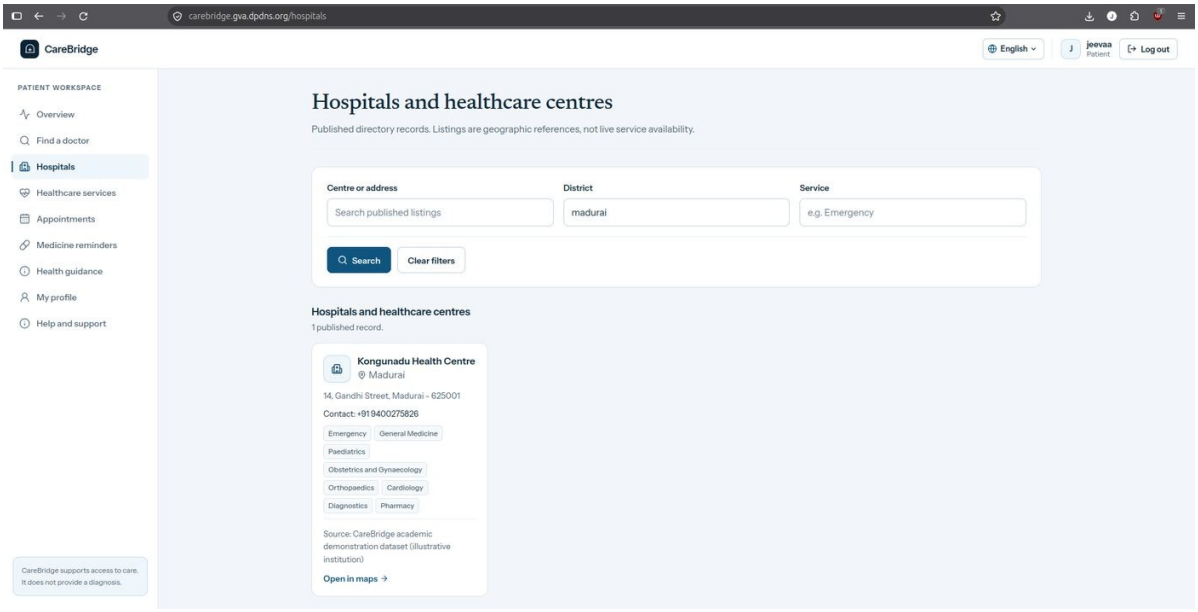


Figure 6.5: Hospitals and healthcare centres

Published directory records filtered by district, showing address, contact, services, data source and an “Open in maps” link.

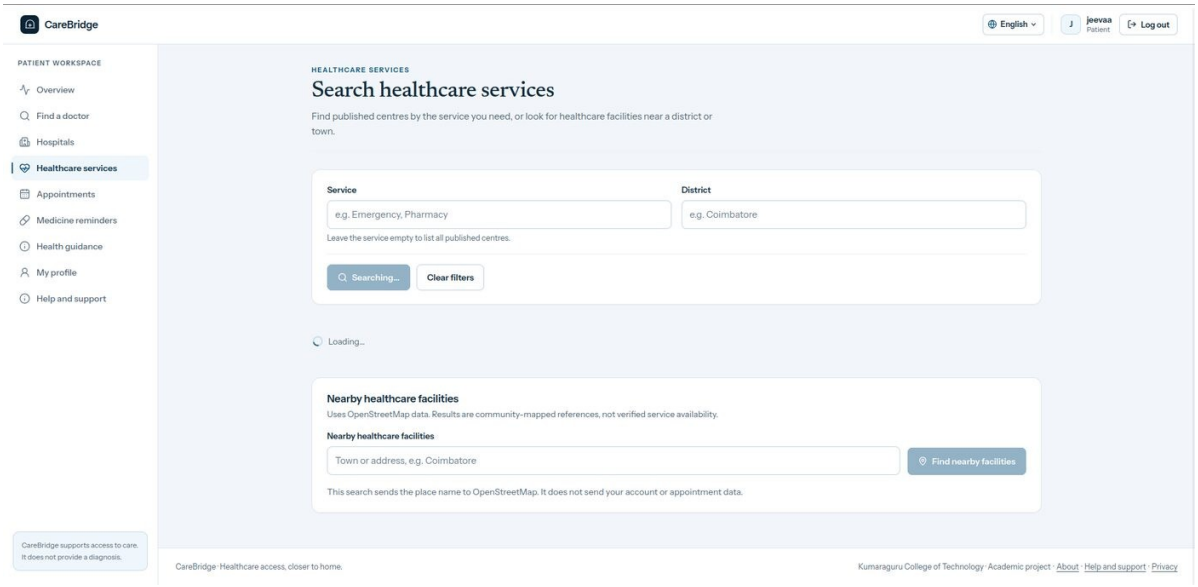


Figure 6.6: Search healthcare services

Patients search published centres by service and district, or look up nearby OpenStreetMap facilities; the notice states that only the place name is sent externally.

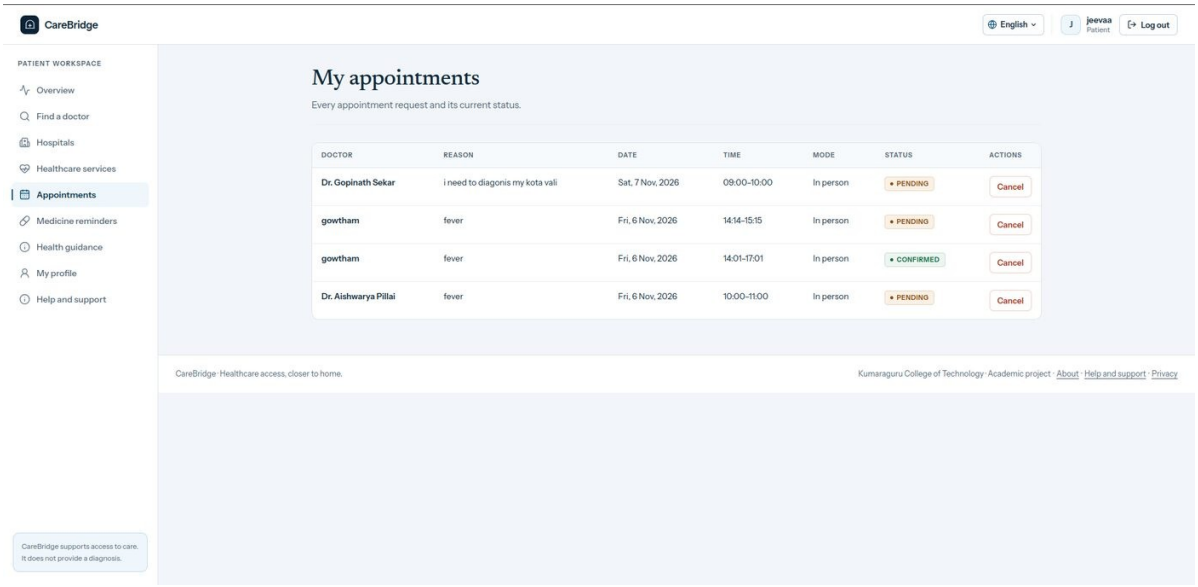


Figure 6.7: My appointments

All of the patient's requests with reason, date, time, mode and colour-coded status; eligible requests can be cancelled.

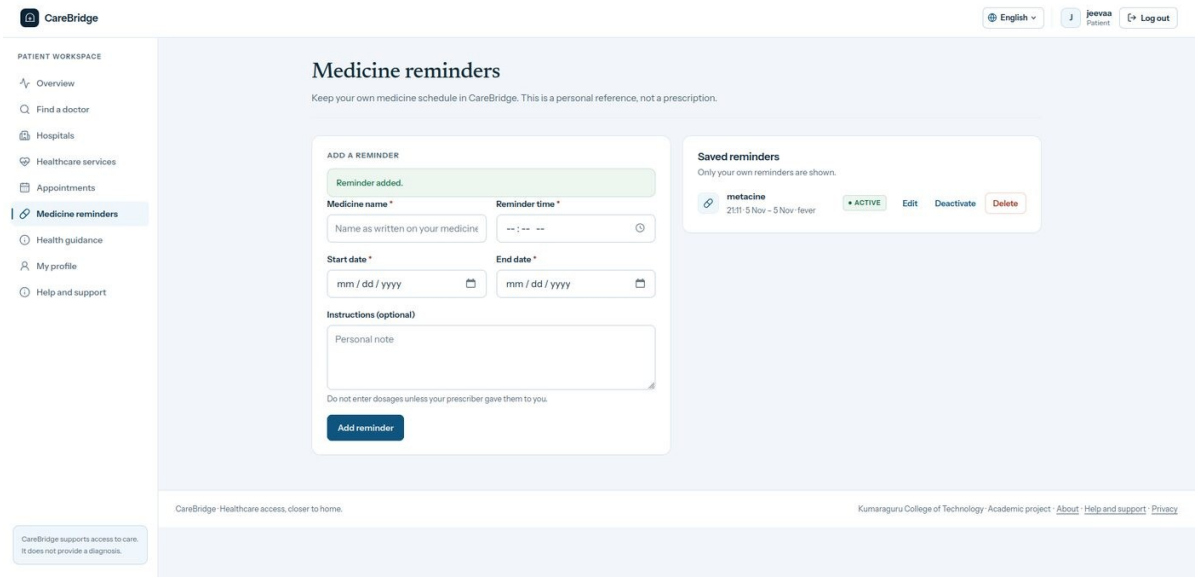


Figure 6.8: Medicine reminders

A form to add a reminder (name, time, start and end date, instructions) and a list of saved reminders with edit, deactivate and delete actions; a note states that CareBridge does not send notifications.

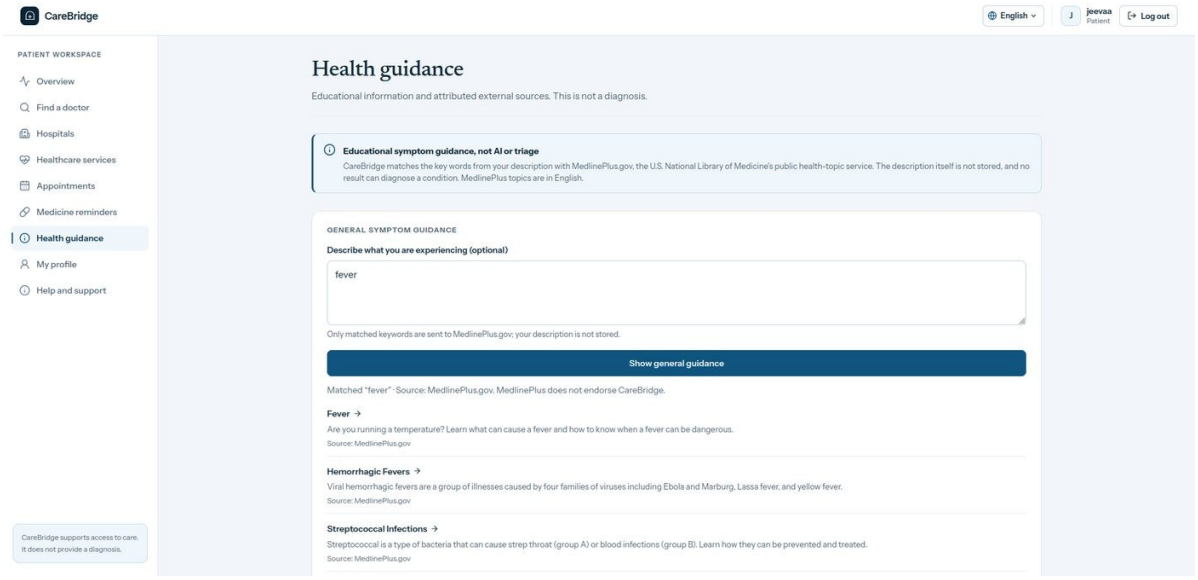


Figure 6.9: Health guidance

Non-diagnostic educational guidance: keywords from the description are matched to attributed MedlinePlus topics, and the description itself is not stored.

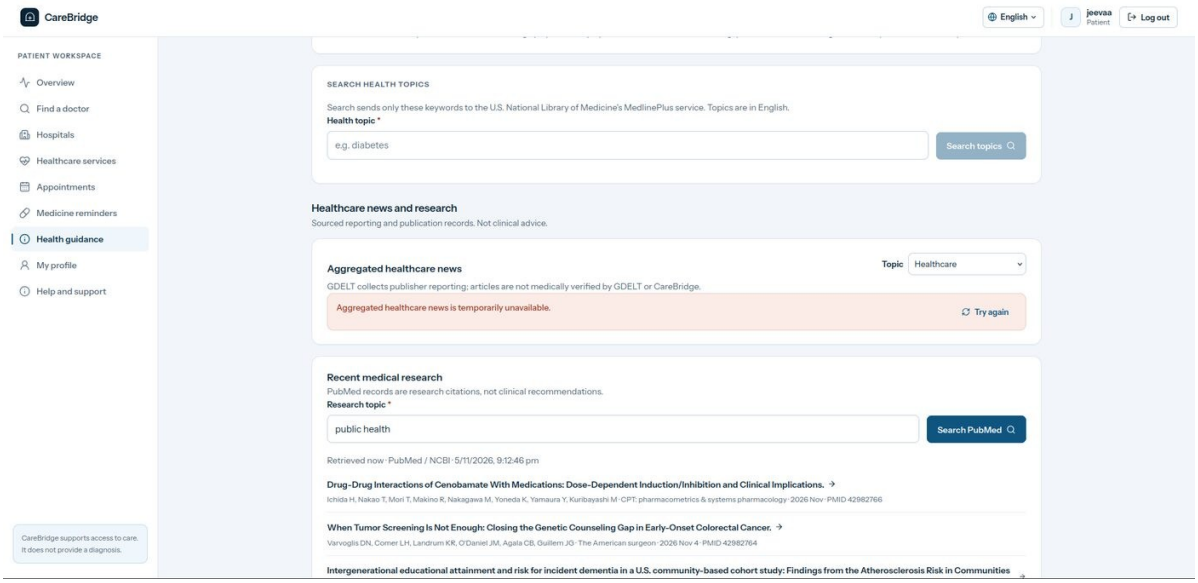


Figure 6.10: Healthcare news and research

Aggregated news and PubMed citations with attribution. The news panel shows a graceful "temporarily unavailable" message with a retry button when the upstream service does not respond.

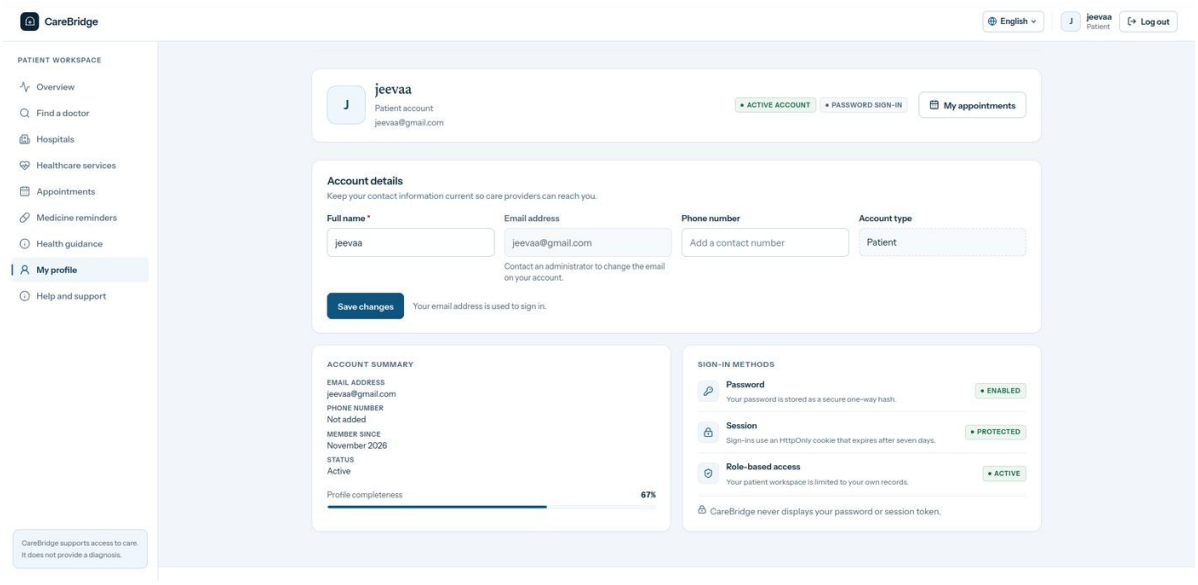


Figure 6.11: Patient profile

Editable account details, profile completeness and sign-in method status (password hash, HttpOnly session, role-based access).

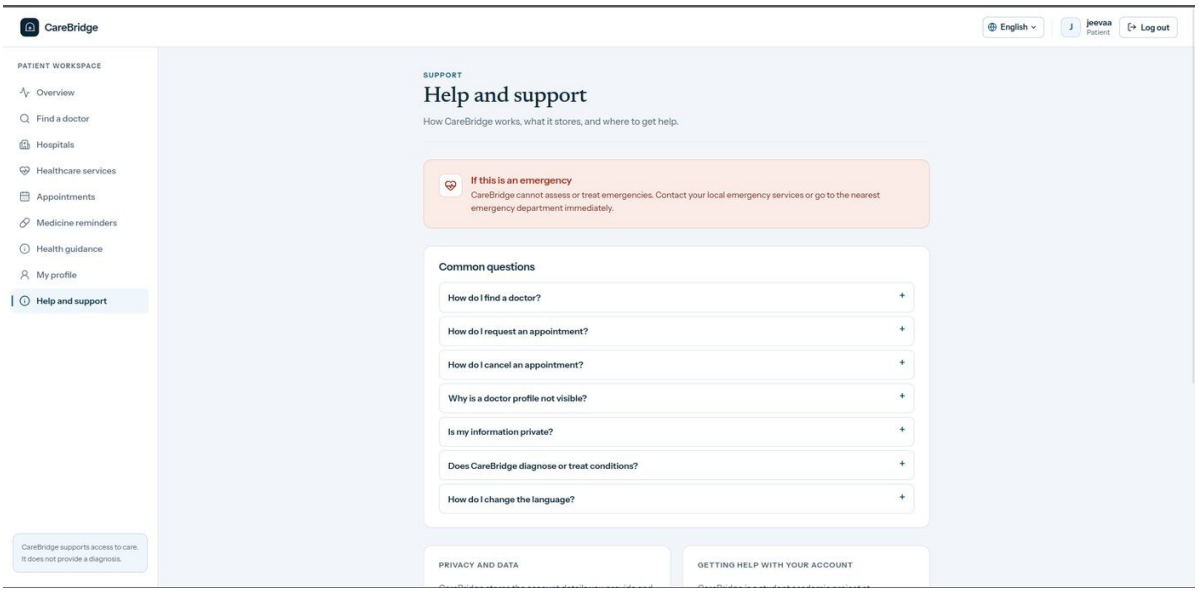


Figure 6.12: Help and support

Emergency guidance, common questions, privacy information and the medical disclaimer.

Doctor screens

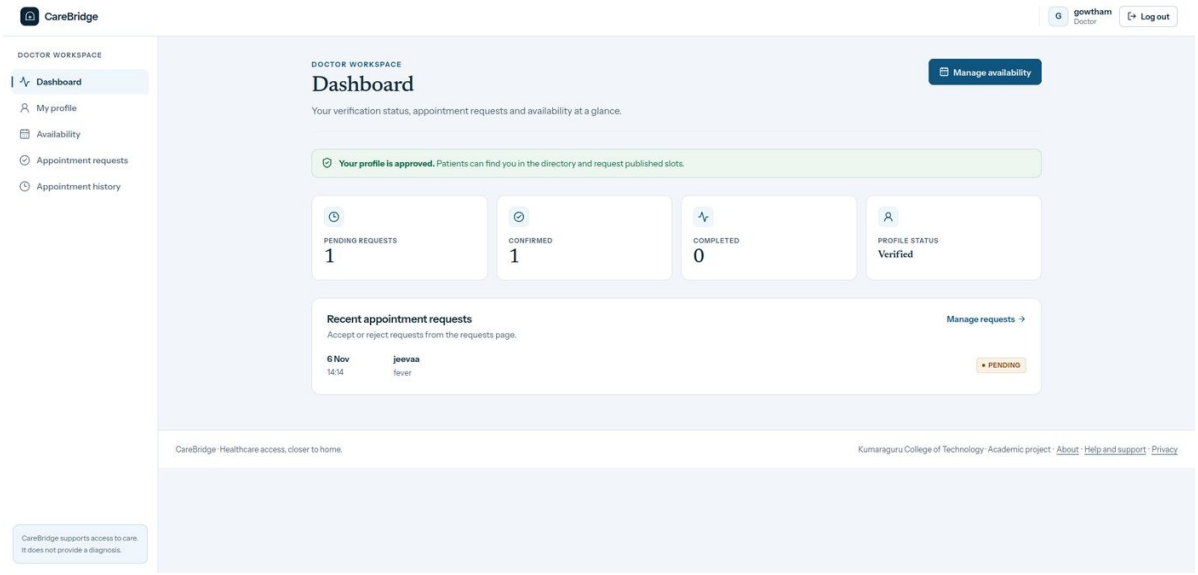


Figure 6.13: Doctor dashboard

Verification status banner, counters for pending, confirmed and completed requests, and recent appointment requests.

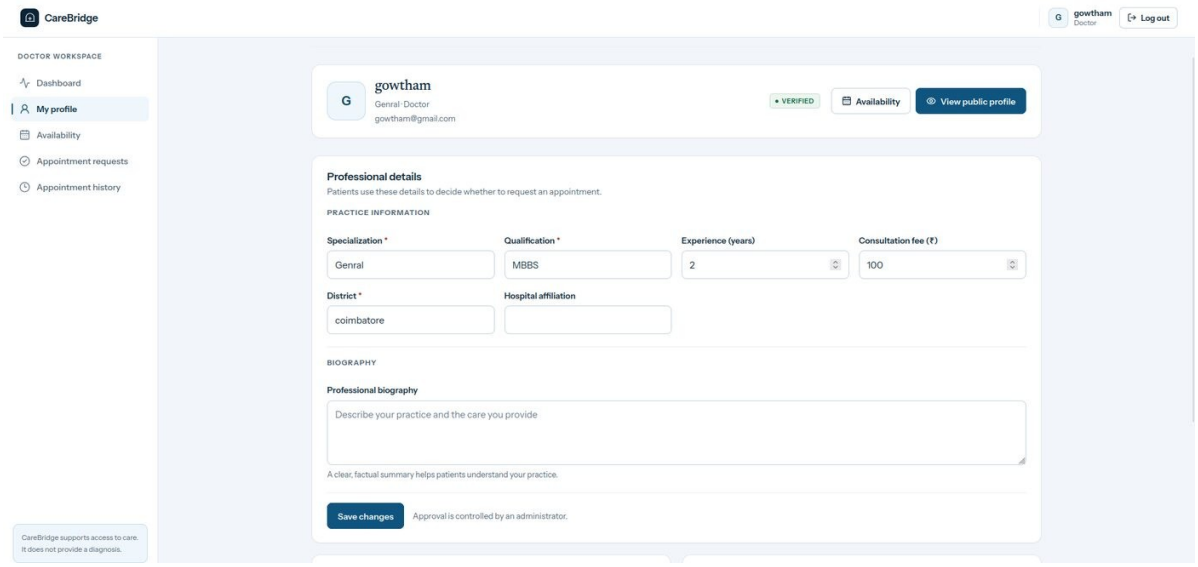


Figure 6.14: Doctor profile editor

Professional details (specialization, qualification, experience, fee, district, biography); approval remains under administrator control.

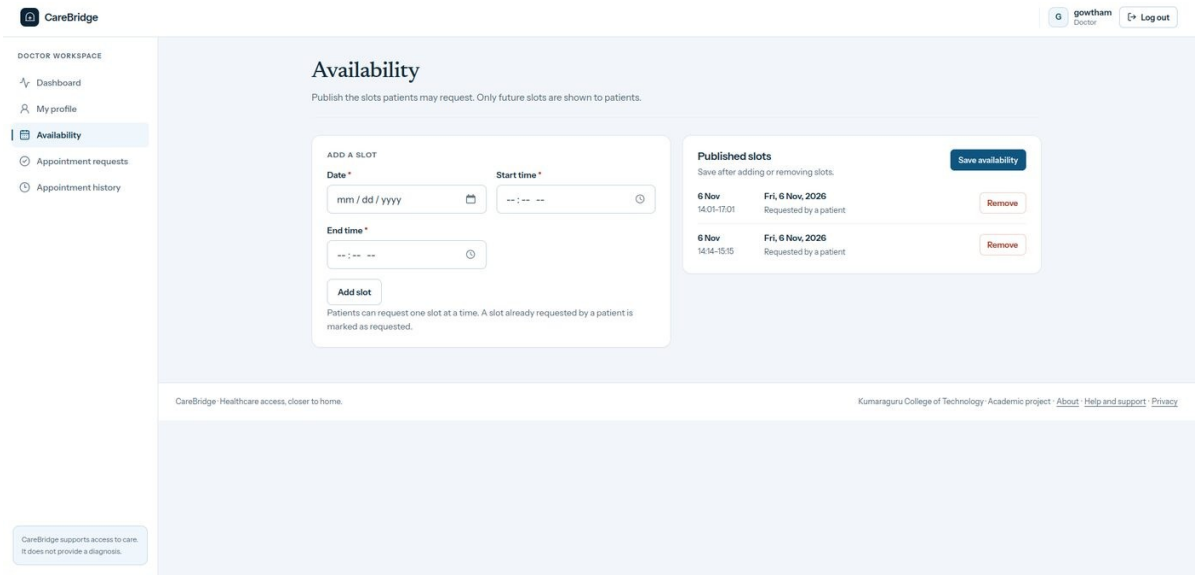


Figure 6.15: Availability management

The doctor adds date and time slots and sees published slots, with a marker when a patient has already requested one.

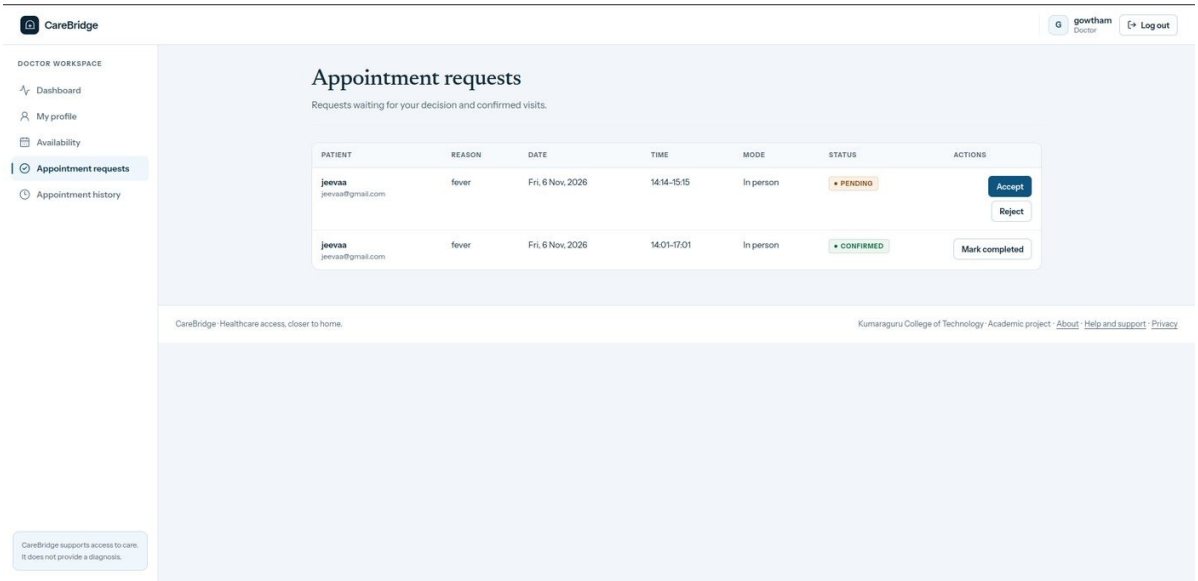


Figure 6.16: Appointment requests

Pending requests can be accepted or rejected; confirmed visits can be marked completed.

Administrator screens

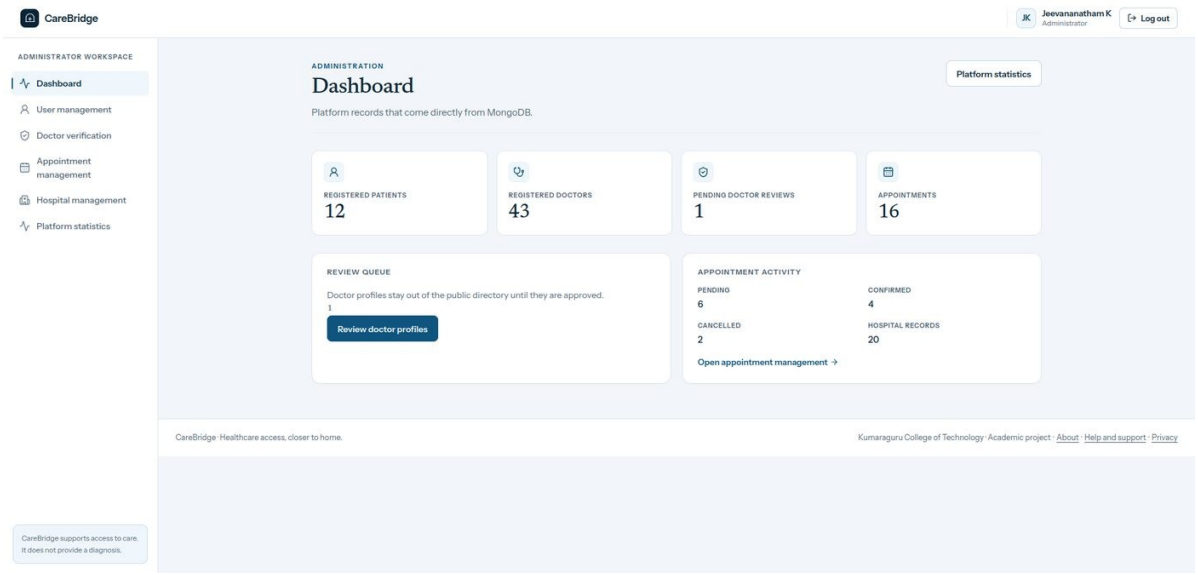


Figure 6.17: Administrator dashboard

Counts of registered patients and doctors, pending doctor reviews, appointments and hospital records, with shortcuts to the review queue.

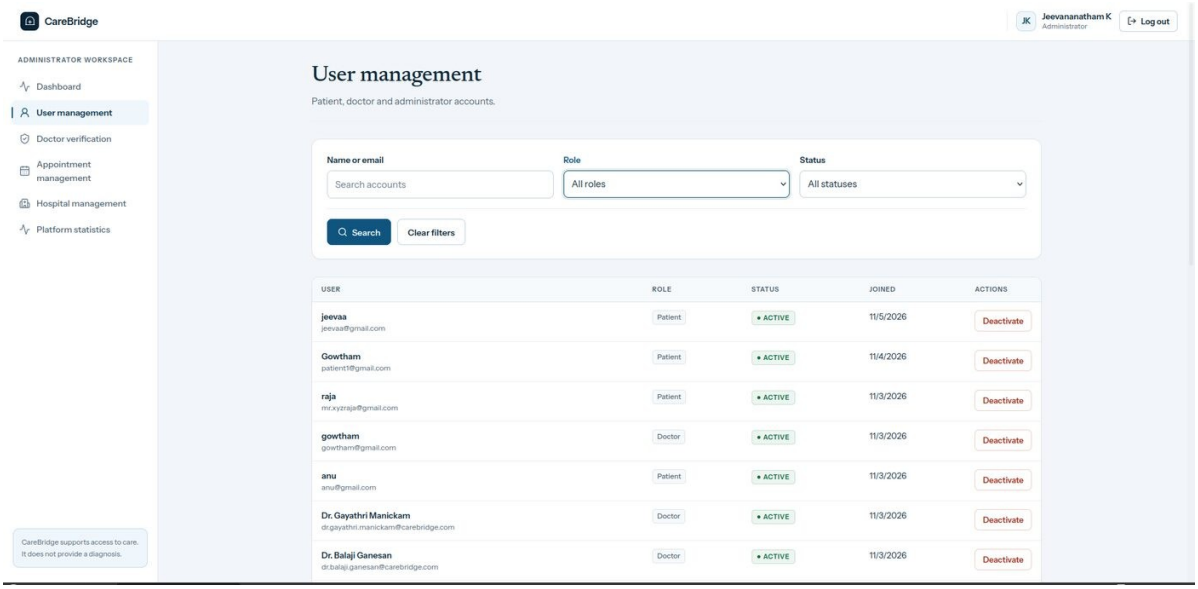


Figure 6.18: User management

Searchable account list filtered by role and status, with Activate/Deactivate actions.

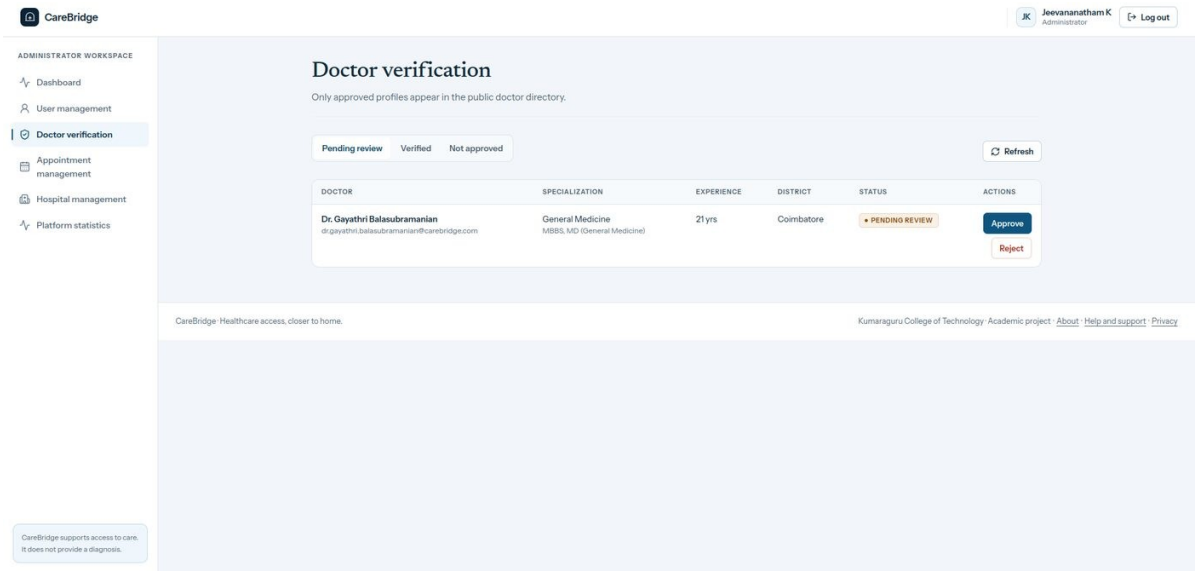


Figure 6.19: Doctor verification

Pending doctor profiles with specialization, experience and district; the administrator approves or rejects each one.

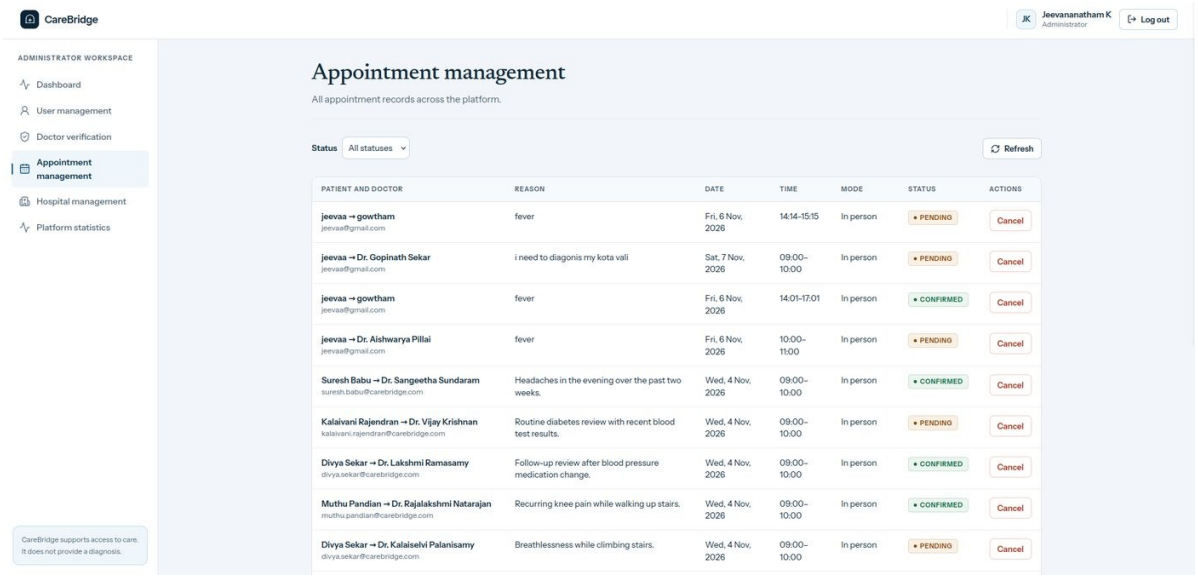


Figure 6.20: Appointment management

All appointments across the platform, filterable by status, with administrator cancel action.

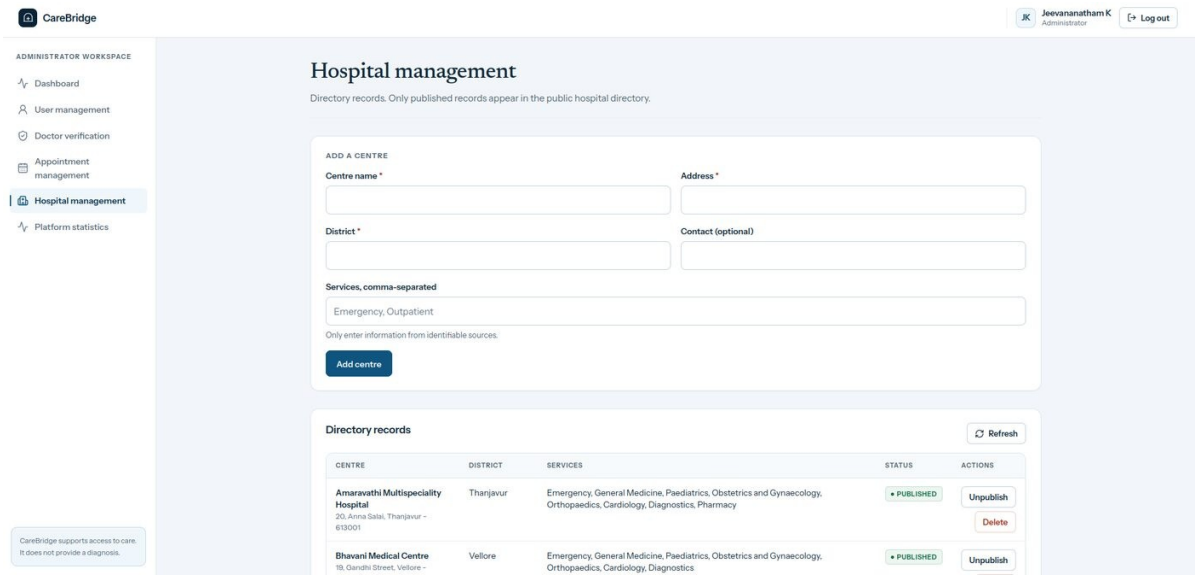


Figure 6.21: Hospital management

A form to add centres and a table of directory records with publish/unpublish and delete actions.

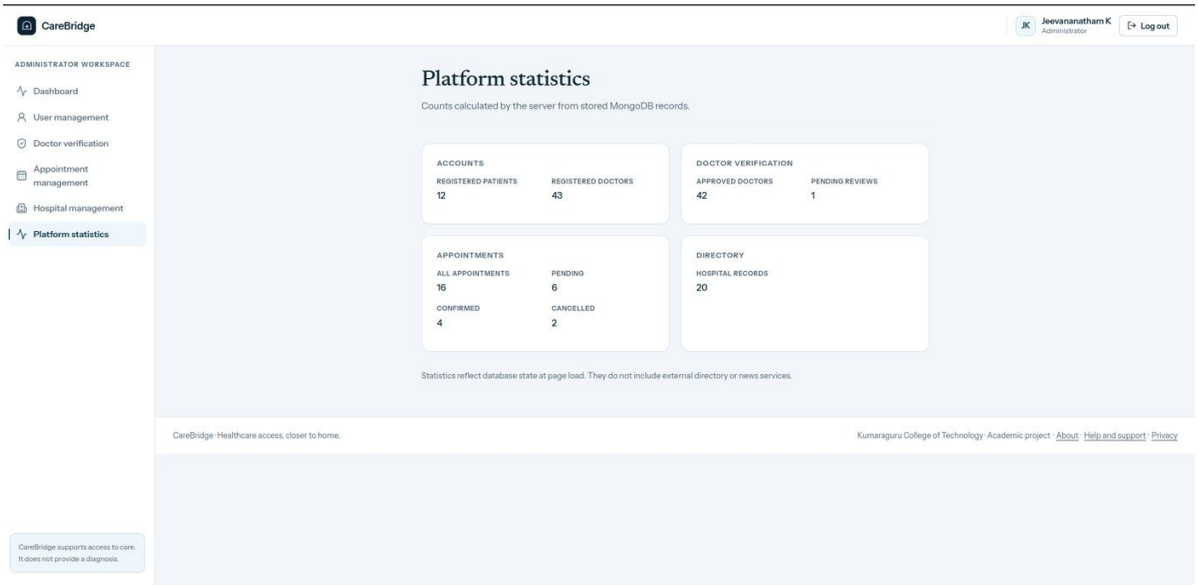


Figure 6.22: Platform statistics

Server-calculated counts for accounts, doctor verification, appointments and the hospital directory.

The layout is responsive: below 900 px the sidebar becomes an off-canvas drawer and cards and tables stack (Section 4.2). The screenshots above were captured at desktop width.

6.2 Objectives Achieved

Table 6.1: Status of objectives from Section 1.3

Objective	Status	Remarks
1. Secure access	Achieved	bcrypt, HttpOnly JWT cookie, rate-limited auth; admin created only by seed script.

Objective	Status	Remarks
2. Verified directory	Achieved	Pending doctors and unpublished hospitals are absent from public queries; covered by automated tests.
3. Reliable booking	Achieved	Partial unique index plus transition map; duplicate-slot and privacy tests pass.
4. Personal adherence support	Achieved	Full CRUD with ownership checks. Reminders are stored and shown but do not trigger notifications.
5. Administration	Achieved	Verification, user status, hospital CRUD, statistics and appointment list working (Figures 6.17–6.22).
6. Accessible and deployable	Partially achieved	Live on Vercel with documented setup. Tamil covers core interface strings only; a formal accessibility audit and mobile-device test were not performed.

6.3 Performance Observations

No dedicated load test or Lighthouse audit was carried out, so the following are the measurements actually recorded during development and deployment.

Table 6.2: Recorded measurements

Measurement	Observation
Client production build (Vite)	2,092 modules transformed in 3.34 s; JavaScript 501 kB (153 kB gzip), CSS 46 kB (12.6 kB gzip) at final validation on 3 Nov 2026.
Vercel production builds	Recent deployments finished in roughly 22–37 s each (Figure 5.2).
Vercel runtime (6 h window)	Error rate 0 % with 139 CDN requests and 91 function invocations (Figure 5.1).
API integration suite	11 tests passed in about 41 s against a live MongoDB at final validation.
Production smoke test	Health, database health, SPA routes and static assets returned HTTP 200; protected endpoints returned HTTP 401 when unauthenticated.
Dependency audit	`npm audit` reported 0 vulnerabilities for server and client at final validation.

The main JavaScript bundle is marginally above Vite’s 500 kB advisory threshold because the client is largely one module; route-level code splitting is listed as future work.

6.4 Challenges Faced

- **Double booking under concurrency.** A “check then insert” approach has a race condition. It was replaced by a partial unique index so MongoDB makes the decision atomically.
- **Express on a serverless platform.** Vercel imports the app instead of running `start()`, which left the database permanently disconnected at first. A lazy, cached connection opened from the request path solved it.

- **Deep links returning 404 after refresh.** Routes such as `/patient` failed on reload until rewrite rules were added to `vercel.json` (commit “Fix routing and logout 404 error”).
- **Test isolation.** Two early tests assumed an empty database and failed against a populated one; they were rewritten to assert the presence or absence of their own synthetic records.
- **Unreliable external services.** News and nearby-facility sources may be slow or unavailable, so every external call is time-bounded, cached and shown with a retry message instead of breaking the page.
- **Team coordination.** Four members worked on one repository; documentation in `docs/` kept conventions for the API, database and security rules consistent.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

CareBridge demonstrates a complete MERN application that addresses a real coordination problem in rural healthcare. Patients can find administrator-verified doctors, request exact slots and track their status; doctors manage their profile, availability and requests; and administrators control what is publicly visible. Across the stack the team applied React state, routing and context; Express middleware, validation and role-based authorisation; MongoDB schema design with partial unique indexes; cookie-based JWT sessions; automated testing; and continuous deployment on Vercel with MongoDB Atlas.

The most important lessons were that security must be enforced on the server rather than in the interface, that integrity rules belong in the database, and that limitations should be stated honestly. All six objectives were met, with Tamil coverage, browser-level accessibility testing and notification delivery recorded as limitations.

7.2 Future Scope

1. **Notifications:** consent-based e-mail, SMS or push reminders for medicines and appointments using a background job scheduler.
2. **Real-time features:** Socket.io for live slot availability and instant status updates.
3. **Teleconsultation:** secure video consultation for the online mode, which is currently a request flag only.
4. **Wider language support and accessibility:** full Tamil and additional regional languages for every message, and a formal WCAG audit with browser-based end-to-end tests.
5. **Performance and analytics:** route-level code splitting, an administrator analytics dashboard on the existing aggregation endpoints, and calendar-style slot management.

7.3 Individual Contribution

The responsibilities below follow the team's documented role allocation (docs/TEAM.md); percentages reflect an equal share of the work.

Table 7.1: Individual contribution of team members

Team Member	Modules / Tasks Handled	Contribution (%)
Akhilon S (24BAD008)	Team lead; application integration, overall review, deployment coordination	25
Jeevanandham K (24BAD047)	React interface and patient-facing workflows; Vercel and GitHub deployment	25
Jeevadharan G (24BAD046)	Express APIs, authentication and MongoDB integration	25
Gowtham S (24BAD028)	Testing, documentation and administrator workflows	25

7.4 References

1. MongoDB Inc., “MongoDB Manual: Partial Indexes,” https://www.mongodb.com/docs/manual/core/index-partial/
2. Mongoose documentation, https://mongoosejs.com/docs/
3. Express.js documentation, https://expressjs.com/
4. React documentation and React Router documentation, https://react.dev/ and https://reactrouter.com/
5. OWASP, “JSON Web Token Cheat Sheet” and “Password Storage Cheat Sheet,” https://cheatsheetseries.owasp.org/
6. U.S. National Library of Medicine, MedlinePlus web service, https://medlineplus.gov/
7. Vercel documentation, https://vercel.com/docs